Cognitive Kernel v1.7

Meta-Substrate & Physics-GovernanceIntegration Specification

Full System Specification / Epoch Engine / Physics-Threat Governance / Substrate-Agnostic Runtime

---

Version: 1.7.0-RELEASE

Document ID: CK-SPEC-1.7.0

Date: June 2026

Classification: Technical Reference

Authors: CK Architecture Council

Supersedes: CK-SPEC-1.6.x

RATIFIED

Document Metadata

Field	Value
Document ID	CK-SPEC-1.7.0
Supersedes	CK-SPEC-1.6.x
Authors	CK Architecture Council

Field	Value
Review Cycle	Quarterly
Applies To	All CK Runtime Instances v1.7+
Hash (SHA-256)	[COMPUTED AT PUBLISH]
License	Internal Specification — Restricted
Document Status	RATIFIED
Classification	Technical Reference
Effective Date	June 2026
Next Scheduled Review	September 2026
Distribution	CK Runtime Implementers, Architecture Leads, Compliance Officers

Document Notice

This specification is an internal technical reference document. Portions marked NORMATIVE constitute binding implementation requirements for all conformant CK v1.7 runtime instances. Portions marked INFORMATIVE are provided for contextual guidance only. Redistribution outside the CK Architecture Council's designated distribution list is prohibited without written authorization.

Table of Contents

---

§0 — Abstract & Document Scope

§0.1 Purpose

This document constitutes the complete, ratified specification for Cognitive Kernel version 1.7.0 (hereafter **CK v1.7**). It defines the architecture, interfaces, protocols, invariants, and operational requirements for all conformant CK runtime instances at version 1.7 and above. This specification is normative for implementers and supersedes all prior CK specification documents in the 1.6.x series.

CK v1.7 represents a major architectural evolution of the Cognitive Kernel platform, introducing four primary systemic advancements:

1. **Meta-Substrate Layer (MSL):** A new architectural stratum providing a uniform, substrate-agnostic abstraction over heterogeneous physical and logical execution substrates, including quantum and post-quantum lattice environments.
2. **Physics-Threat Governance (PhysicsThreatAlert):** A formal, real-time threat detection and governance system capable of identifying substrate-level physical anomalies, including coherence collapse, entanglement poisoning, lattice parameter drift, and cross-substrate interference, and triggering appropriate automated and human-mediated responses.
3. **Substrate-Agnostic Epoch Engine:** A fully redesigned epoch lifecycle engine decoupled from any specific physical substrate, enabling deterministic epoch transitions, formal rollback guarantees, and portable checkpointing across all MSL-registered substrates.
4. **Consensus Contract v1.7:** An upgraded consensus framework incorporating formal invariants, enhanced quorum rules, and machine-verifiable contract terms, replacing the informal consensus guidelines of v1.6.3.

The architectural goals of CK v1.7 are summarized as follows:

- **Substrate Independence:** The CK runtime MUST operate correctly across any substrate that satisfies the Substrate Abstraction Interface (SAI), without requiring substrate-specific modifications to core runtime logic.
- **Formal Physics Invariants:** All interactions between the CK runtime and physical or logical substrates MUST be governed by formally stated, machine-checkable invariants to prevent undefined behavior arising from substrate-level physical phenomena.
- **Deterministic Epoch Transitions:** All epoch lifecycle transitions MUST be deterministic, auditable, and reproducible given identical inputs and substrate state snapshots.
- **Governance Auditability:** All governance decisions, including consensus outcomes, physics-threat responses, and design-law enforcement actions, MUST be recorded in an immutable, indexed audit log suitable for post-hoc verification.

§0.2 What Changed from v1.6

CK v1.6.x established the foundational Cognitive Kernel architecture, including the basic epoch engine, a preliminary consensus contract (v1.6.3), and the Internal Event Bus (IEB). However, v1.6 suffered from several architectural limitations:

- The epoch engine was tightly coupled to a single physical substrate type, preventing portability.
- No substrate abstraction layer existed; substrate-specific code was embedded directly in runtime modules.
- Physics-level threat conditions (e.g., decoherence events on quantum substrates) had no formal detection or response pathway.
- The consensus contract relied on informal documentation rather than machine-enforceable invariants.
- Design laws were expressed as advisory guidelines with no enforcement mechanism.

CK v1.7 resolves all of the above limitations. The version delta is presented in full detail in §3.

---

§1 — Introduction & Design Philosophy

1.1 Background — Evolution from CK v1.0 to v1.7

The Cognitive Kernel project was initiated to provide a principled, formal substrate for high-integrity cognitive computation across diverse execution environments. The evolutionary trajectory of the CK platform is summarized below:

Version	Key Milestone	Year
CK v1.0	Initial release. Single-substrate classical runtime. Basic epoch tracking. No formal governance.	2020
CK v1.1	Introduced Internal Event Bus (IEB) in preliminary form. Added consensus concept as informal protocol.	2021
CK v1.2	Epoch Engine v1 released. Added epoch rollback (best-effort, non-deterministic).	2021
CK v1.3	IEB formalized. Audit log introduced. Consensus Contract v1.3 drafted.	2022
CK v1.4	First experimental quantum substrate integration (non-abstracted). Design Law guidelines introduced informally.	2023
CK v1.5	Performance optimizations. Consensus Contract v1.5 with quorum rules. Epoch checkpointing (basic).	2023
CK v1.6	Consensus Contract v1.6.3. Improved audit logging. Substrate coupling remains.	2024–2025

Version	Key Milestone	Year
	No formal physics governance.	
CK v1.7	Meta-Substrate Layer. PhysicsThreatAlert. Substrate-agnostic Epoch Engine. Formal Design Law. Consensus Contract v1.7.	2026

1.2 Core Principles

CK v1.7 is governed by four foundational design principles. These principles are not aspirational; they are architectural constraints that all conformant implementations MUST satisfy.

1.2.1 Determinism

Given an identical initial state vector and an identical ordered sequence of inputs, any two conformant CK v1.7 runtime instances MUST produce identical outputs and identical intermediate state transitions. Determinism applies across epoch boundaries, substrate transitions, and governance events. Non-deterministic behavior is a conformance failure.

1.2.2 Substrate Agnosticism

The CK runtime core MUST NOT contain substrate-specific logic. All substrate interactions MUST be mediated exclusively through the Meta-Substrate Layer (MSL) and the Substrate Abstraction Interface (SAI). This principle ensures that the runtime operates correctly on any substrate satisfying the SAI contract, whether classical, quantum, post-quantum lattice, or future substrate types.

1.2.3 Physics Invariance

The CK runtime MUST enforce formally stated invariants at the boundary between the MSL and physical substrates. These invariants capture the physical constraints of each substrate class (e.g., coherence window bounds for quantum substrates, lattice parameter stability bounds for post-quantum substrates) and MUST be checked continuously by the PhysicsThreatAlert subsystem. Invariant violations MUST trigger defined response actions.

1.2.4 Governance Auditability

Every governance decision made by a CK v1.7 runtime instance MUST be recorded with sufficient fidelity to permit independent post-hoc verification. Audit records MUST be tamper-evident, indexed by epoch and timestamp, and retained for a minimum period as specified in §12.5.

1.3 Scope of This Specification

This specification defines:

- The architecture and interfaces of the Meta-Substrate Layer (§4).
- The definitions and compliance requirements for QIGEM_v1 and PostLattice_v2 substrates (§5).
- The substrate-agnostic Epoch Engine design, state machine, and invariants (§6).
- The Physics-Threat Alert System architecture, schemas, and response policies (§7).
- The Consensus Contract v1.7 terms, formal invariants, and quorum rules (§8).
- The Internal Event Bus v1.7 additions, schemas, and delivery semantics (§9).
- The formal Design Law encoding, law hierarchy, and enforcement mechanisms (§10).
- The migration guide and toolchain for upgrading from CK v1.6 to CK v1.7 (§11).
- Observability, testing, and compliance requirements (§12).
- The CK v1.8 roadmap preview (§13).

This specification does NOT define application-layer behavior above the Consensus Layer, specific hardware implementation details for substrates, or network-layer protocols used for inter-node communication (except where those protocols intersect with the IEB).

1.4 Normative vs. Informative Sections (RFC 2119 Terms)

The key words in this specification are used as defined in RFC 2119 (Bradner, 1997):

Keyword	Meaning
MUST / **REQUIRED** / **SHALL**	An absolute requirement of this specification. Non-compliance constitutes a conformance failure.
MUST NOT / **SHALL NOT**	An absolute prohibition of this specification.
SHOULD / **RECOMMENDED**	A strong recommendation. Deviation is permitted with documented justification, but carries interoperability or safety risk.
SHOULD NOT / **NOT RECOMMENDED**	A strong recommendation against. Deviation is permitted with documented justification.
MAY / **OPTIONAL**	Truly optional. Implementations that do not include the feature MUST still interoperate with implementations that do.

Sections labeled **[NORMATIVE]** contain binding requirements. Sections labeled **[INFORMATIVE]** provide contextual guidance only. Unlabeled sections should be interpreted as normative by default.

1.5 Relationship to External Standards

CK v1.7 references or is aligned with the following external standards and documents:

Standard / Document	Relevance
RFC 2119 (BCP 14)	Keyword interpretation for normative requirements.
NIST SP 800-208	Lattice-based cryptography recommendations; informs PostLattice_v2 substrate design.
NIST PQC Standards (FIPS 203, 204, 205)	Post-quantum cryptographic primitives referenced in PostLattice_v2 hardness assumptions.
IEEE 7000-2021	Model process for addressing ethical concerns during system design; informs Design Law framework.

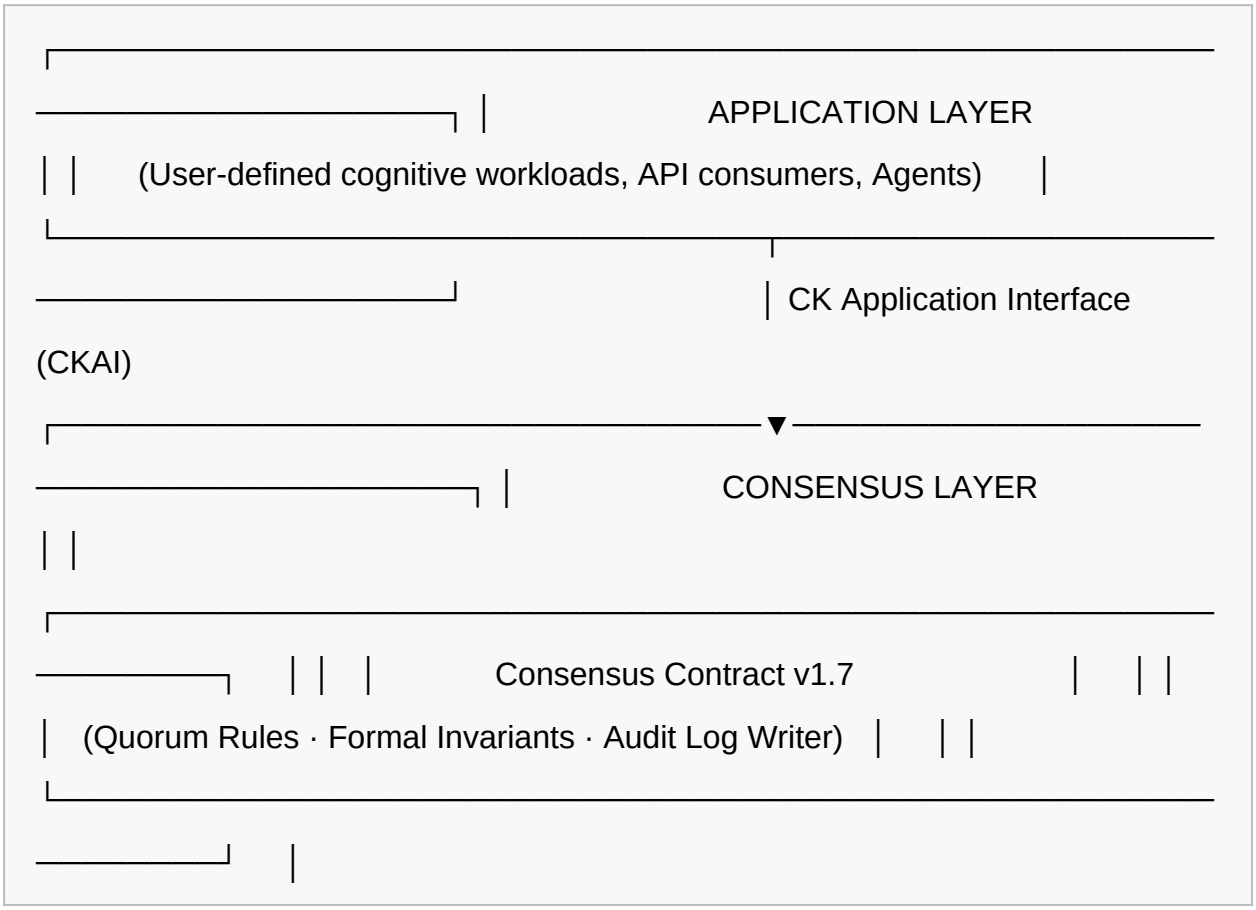
Standard / Document	Relevance
ISO/IEC 25010:2023	Systems and software quality model; informs observability and compliance requirements in §12.
W3C PROV-DM	Provenance data model; informs audit log schema design.

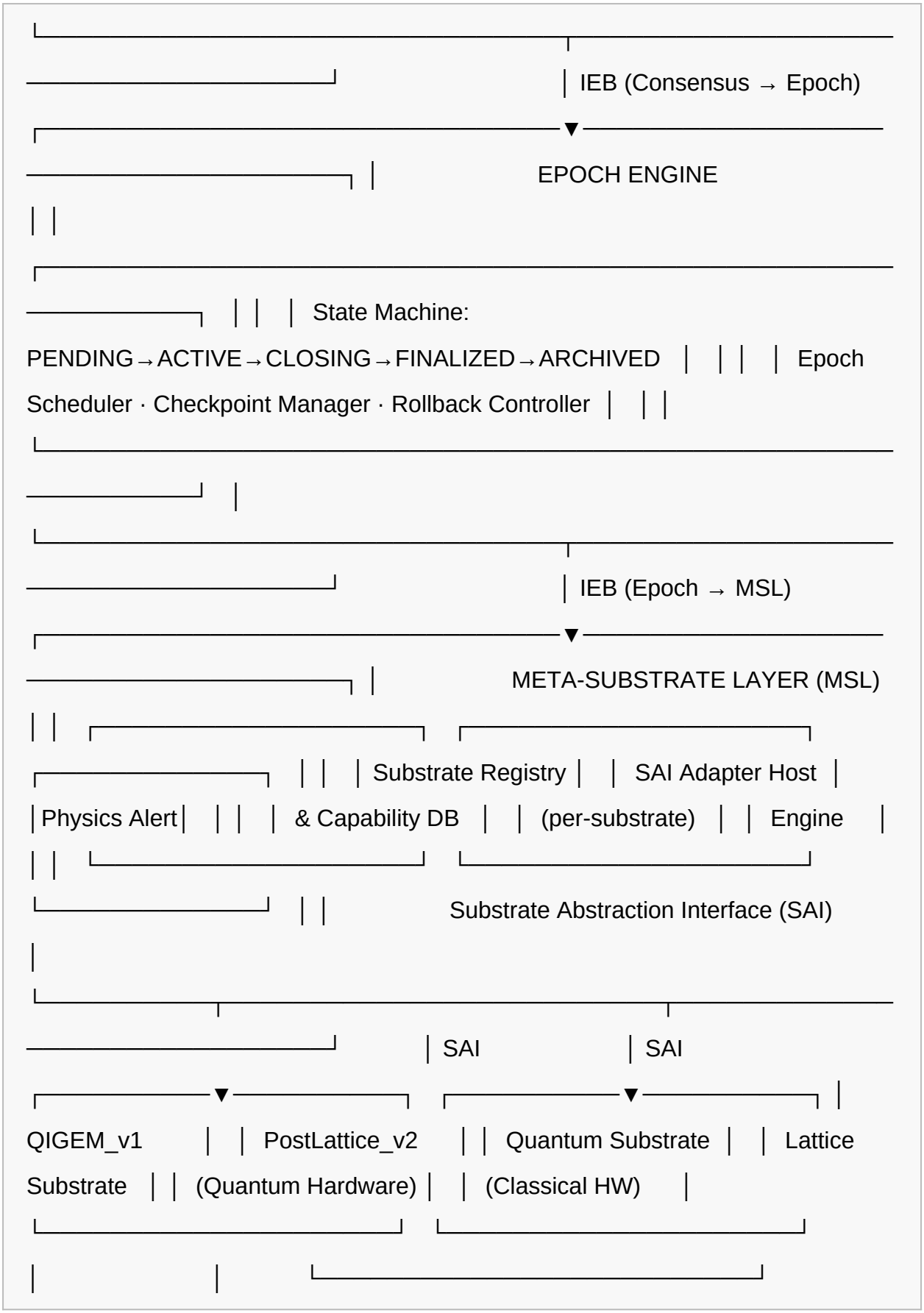
---

§2 — Architecture Overview

2.1 High-Level Component Diagram

The following ASCII diagram depicts the high-level component architecture of a CK v1.7 runtime instance. All inter-layer communication is mediated by the Internal Event Bus (IEB). The Meta-Substrate Layer (MSL) forms the critical boundary between the substrate-agnostic CK core and physical execution substrates.




IEB (Consensus → Epoch)
EPOCH ENGINE
State Machine:
PENDING→ACTIVE→CLOSING→FINALIZED→ARCHIVED
Epoch Scheduler · Checkpoint Manager · Rollback Controller
IEB (Epoch → MSL)
META-SUBSTRATE LAYER (MSL)
Substrate Registry & Capability DB
SAI Adapter Host (per-substrate)
Physics Alert Engine
Substrate Abstraction Interface (SAI)
SAI
QIGEM_v1
SAI
PostLattice_v2
Quantum Substrate (Quantum Hardware)
Lattice Substrate (Classical HW)

PHYSICAL SUBSTRATE LAYER

2.2 Layer Stack

The CK v1.7 architecture is organized as a strictly layered stack. Each layer communicates only with its immediate neighbors, except for cross-cutting concerns (audit log, IEB) which span multiple layers.

Layer	Responsibility	Key Component(s)
Application Layer	Hosts user-defined workloads and agents. Consumes the CK Application Interface.	CK Application Interface (CKAI)
Consensus Layer	Manages distributed agreement, quorum, and conflict resolution across CK instances.	Consensus Contract v1.7
Epoch Engine	Controls the lifecycle of computational epochs: scheduling, transitions, checkpointing, rollback.	Epoch State Machine, Checkpoint Manager
Meta-Substrate Layer	Abstracts physical substrate heterogeneity. Hosts the SAI adapter for each registered substrate. Runs PhysicsThreatAlert.	MSL, SAI, Substrate Registry, PhysicsThreatAlert
Physical Substrate Layer	The actual execution medium (quantum hardware, lattice-based classical hardware, future substrates).	QIGEM_v1, PostLattice_v2, future substrates

2.3 Internal Event Bus (IEB) — Architecture and Message Flow

The Internal Event Bus (IEB) is the sole communication channel between all CK layers. All inter-layer messages MUST be expressed as typed, versioned IEB events. Direct function calls or shared memory between layers are **MUST NOT** be used.

The IEB provides the following delivery guarantees, configurable per event class:

- **At-most-once:** Suitable for advisory telemetry events where loss is acceptable.
- **At-least-once:** Suitable for governance notifications requiring receipt confirmation.
- **Exactly-once:** Required for consensus and epoch transition events where duplicate delivery would cause state corruption.

Full IEB v1.7 additions are defined in §9.

2.4 Key Interfaces and Abstraction Boundaries

Interface	Between Layers	Protocol	Notes
CKAI	Application ↔ Consensus	IEB event stream	Application-facing; versioned.
Consensus–Epoch Interface	Consensus ↔ Epoch Engine	IEB event stream	Triggers epoch lifecycle events from consensus outcomes.
Epoch–MSL Interface	Epoch Engine ↔ MSL	IEB event stream	Carries epoch commands to MSL for substrate dispatch.
SAI	MSL ↔ Physical Substrate	SAI adapter call	Typed, versioned; substrate-specific adapters implement SAI.
PhysicsThreatAlert Bus	MSL → Consensus / Application	IEB priority channel	High-priority, pre-empts normal event queue.

2.5 Runtime Lifecycle Overview

A CK v1.7 runtime instance progresses through the following top-level lifecycle phases:

5. **BOOTSTRAP:** Runtime initialization. MSL loads and validates registered substrates via SAI. PhysicsThreatAlert subsystem starts monitoring. IEB initializes.

6. **SUBSTRATE_READY:** All required substrates have passed SAI validation. Epoch Engine initializes in PENDING state.
7. **OPERATIONAL:** Epoch Engine enters ACTIVE state. Application Layer begins accepting workloads. Consensus Contract active.
8. **GOVERNED_PAUSE:** Triggered by a CRITICAL PhysicsThreatAlert or governance event. Application workloads suspended. Governance response executes.
9. **SUBSTRATE_HALT:** Triggered by a SUBSTRATE_HALT alert. Full runtime suspension. Human-in-the-loop escalation mandatory.
10. **SHUTDOWN:** Orderly termination. Final epoch finalized, audit log flushed, substrates deactivated via MSL.

---

§3 — Version Delta: v1.6 → v1.7

The following table documents all architectural changes between CK v1.6 (latest: v1.6.3) and CK v1.7.0-RELEASE. Change types are classified as: **Breaking** (requires mandatory adaptation), **Additive** (new capability; backward-compatible), or **Deprecated** (existing feature marked for removal in v1.8).

Component	v1.6 Behavior	v1.7 Behavior	Change Type
Epoch Engine	Substrate-coupled. Epoch lifecycle management embedded substrate-specific logic for the single classical substrate. Epoch transitions not formally defined.	Substrate-agnostic. All substrate interactions delegated to MSL via SAI. Epoch state machine formally defined with typed guards and transitions.	**Breaking**
Meta-Substrate Layer (MSL)	Not present. No substrate abstraction existed. Substrate-specific code embedded in Epoch Engine and Consensus Layer.	Introduced. Provides Substrate Abstraction Interface (SAI), Substrate Registry, Capability Matrix enforcement, and per-substrate SAI adapter hosting.	**Additive (required)**

Component	v1.6 Behavior	v1.7 Behavior	Change Type
PhysicsThreatAlert	Not present. Physics-level substrate anomalies were not detected or governed. Coherence events caused undefined behavior.	Introduced. Full threat taxonomy defined (§7.2). Alert pipeline, schema, severity levels, routing, and human-in-the-loop escalation policy all defined.	**Additive (required)**
Consensus Contract	v1.6.3. Informal documentation. Quorum rules partially specified. No machine-verifiable invariants. No formal conflict resolution protocol.	v1.7. Formal invariants defined (§8.4). Quorum rules fully specified and machine-enforceable. Conflict resolution protocol formalized. Audit log requirements mandated.	**Breaking**
Internal Event Bus (IEB)	Basic event types. Schema versioning not enforced. Delivery semantics per-event class not defined. No dead letter queue. No backpressure mechanism.	Extended schema with mandatory versioning. Delivery semantics explicitly defined per event class. Dead letter queue policy defined (§9.6). Backpressure and flow control specified (§9.5).	**Breaking**
QIGEM_v1 Substrate	Not present. Quantum substrate integration was experimental and non-abstracted in v1.4–v1.6; no formal definition existed.	Formally defined. Full substrate specification including coherence window parameters, gate fidelity requirements, entanglement topology table, SAI compliance specification, and configuration schema (§5.1).	**Additive**
PostLattice_v2 Substrate	Not present as a formal substrate definition. PostLattice_v1 existed as an informal prototype not governed by SAI.	Formally defined. Lattice parameters, hardness assumptions (LWE, Ring-LWE), throughput/latency profile, SAI compliance, upgrade path from PostLattice_v1, and configuration	**Additive**

Component	v1.6 Behavior	v1.7 Behavior	Change Type
		schema fully specified (§5.2).	
Design Law	Informal guidelines. Expressed as advisory wiki documentation. No law hierarchy, no enforcement mechanism, no waiver process.	Formally encoded. Four-level hierarchy (Constitutional → Statutory → Policy → Guideline). Prohibited and approved pattern registries. Machine-enforceable via Design Law Enforcement Engine (DLEE). Waiver process defined (§10.8).	**Breaking**
Migration Toolchain	Not present. Migration between minor versions was manual and undocumented.	Provided. Includes pre-migration checker, substrate adapter generator, IEB schema migrator, and post-migration validation test suite (§11.4).	**Additive**
Epoch Rollback	Best-effort, non-deterministic. No formal rollback protocol. Rollback could leave partial state inconsistencies.	Formal rollback semantics defined (§6.6). Deterministic rollback guaranteed for all epochs with a valid checkpoint. Rollback eligibility conditions formally specified.	**Breaking**
Audit Log	Present but optional. No standardized schema. Retention period not specified. Tamper-evidence not guaranteed.	Mandatory. Standardized schema (§12.5). Minimum retention period specified. Tamper-evidence via hash-chaining required.	**Breaking**
PostLattice_v1 Substrate	Informal prototype. Used in some v1.5–v1.6 experimental deployments.	Deprecated. MUST migrate to PostLattice_v2 using provided upgrade path (§5.2). Will be removed in CK v1.8.	**Deprecated**

---

§4 — Meta-Substrate Layer (MSL)

4.1 Purpose and Motivation

Prior to CK v1.7, the CK runtime contained substrate-specific logic embedded directly within the Epoch Engine and, to a lesser extent, the Consensus Layer. This coupling imposed severe constraints on portability, testability, and the ability to introduce new substrate types without modifying core runtime logic. The Meta-Substrate Layer (MSL) was introduced to eliminate this coupling entirely.

The MSL serves as the exclusive intermediary between the substrate-agnostic CK core (Epoch Engine, Consensus Layer, Application Layer) and the physical or logical execution substrates. Its introduction enables:

- Addition of new substrate types without modifying any component above the MSL.
- Uniform capability negotiation between the runtime and substrates.
- Centralized physics-threat monitoring and response.
- Substrate lifecycle management (registration, validation, activation, deactivation) with formal state guarantees.
- Hot-swap substrate support in future versions (CK v1.8, see §13.2).

4.2 MSL Architecture

The MSL is composed of the following sub-components:

- **Substrate Registry:** Maintains the catalog of all registered substrates, their SAI adapters, and their current lifecycle state.
- **SAI Adapter Host:** Loads and manages the SAI adapter for each registered substrate. Each adapter is isolated and cannot directly access another substrate's adapter or the CK core above the MSL.
- **Capability Negotiator:** Compares the capabilities declared by each substrate's SAI adapter against the Substrate Capability Matrix (§4.5) and

determines whether the substrate satisfies the required capability set for the requested configuration.

- **PhysicsThreatAlert Engine:** Continuously monitors physics-level substrate telemetry and evaluates incoming substrate measurements against defined threat conditions. See §7 for full specification.
- **MSL Event Publisher:** Publishes MSL lifecycle and alert events to the IEB for consumption by upper layers.

4.3 Substrate Abstraction Interface (SAI) — Full Interface Specification

All substrates registered with the MSL MUST implement the Substrate Abstraction Interface (SAI). The SAI is the sole programmatic boundary between the MSL and substrate adapters. The SAI is versioned; CK v1.7 requires SAI version 1.7 or later.

The SAI interface specification is as follows:

```
SAI Interface v1.7    substrate_id        : STRING       [REQUIRED]
Unique identifier for this substrate instance. Format:
[type]_[version]_[instance_uuid].     Example: "QIGEM_v1_a3f7c291"
substrate_type      : ENUM         [REQUIRED]     One of: QUANTUM,
LATTICE_PQ, CLASSICAL, HYBRID. Extensible in future SAI versions.
sai_version         : STRING       [REQUIRED]     SAI version implemented by
this adapter. MUST be "1.7" or later.    capabilities        : CAPABILITY_SET
[REQUIRED]     Typed set of capability declarations. See §4.5 for full
Capability Matrix.    health_probe()      : HEALTH_STATUS [REQUIRED]
Invoked by MSL on a configurable polling interval (default: 1s).     Returns:
{ status: ENUM(OK, DEGRADED, FAILED), metrics: METRIC_MAP, timestamp:
UTC_TIMESTAMP }    execute(op: SUBSTRATE_OP) : EXECUTION_RESULT [REQUIRED]
Dispatches a substrate operation. MUST be synchronous or return a tracked
async handle.     op includes: { op_type, payload, timeout_ms, epoch_id,
trace_id }    checkpoint(epoch_id: STRING) : CHECKPOINT_HANDLE [REQUIRED]
Creates a substrate-level checkpoint for the given epoch. MUST be atomic.
restore(checkpoint_handle: CHECKPOINT_HANDLE) : RESTORE_RESULT [REQUIRED]
Restores substrate state to the given checkpoint. MUST be idempotent.
get_physics_telemetry() : PHYSICS_TELEMETRY [REQUIRED]     Returns current
physics-level measurements relevant to PhysicsThreatAlert evaluation.
Content is substrate-type-specific (see §7.2 for threat taxonomy).
deactivate() : VOID [REQUIRED]     Orderly substrate deactivation. MUST flush
pending operations before returning.    declare_metadata() :
SUBSTRATE_METADATA [REQUIRED]     Returns immutable substrate metadata:
vendor, hw_model, firmware_version, topology_descriptor.
```

4.4 MSL Lifecycle: Registration, Validation, Activation, Deactivation

Each substrate progresses through the following MSL lifecycle states:

11. **REGISTERED:** The substrate adapter has been loaded by the SAI Adapter Host and its identity fields validated. No execution capability available.
12. **VALIDATING:** The Capability Negotiator is executing capability checks. `health_probe()` and `declare_metadata()` are called. Physics telemetry baseline is established for PhysicsThreatAlert.
13. **STANDBY:** Validation passed. Substrate is available but not yet assigned to an epoch. `execute()` MUST NOT be called in STANDBY.
14. **ACTIVE:** Substrate is assigned to one or more epoch contexts and may receive `execute()` calls. PhysicsThreatAlert monitoring is fully active.
15. **SUSPENDED:** Substrate has been suspended due to a governance event (e.g., a CRITICAL PhysicsThreatAlert). `execute()` MUST NOT be called. State may be restored to ACTIVE upon governance clearance.
16. **DEACTIVATED:** `deactivate()` has been called. Substrate adapter is unloaded from the SAI Adapter Host. The substrate record is retained in the Substrate Registry for audit purposes.
17. **FAILED:** Substrate failed validation or experienced an unrecoverable error. MUST NOT be reactivated without a full re-registration cycle.

4.5 Substrate Capability Matrix

Capability	Required?	Optional?	QIGEM_v1	PostLattice_v2	Notes
EXECUTE_BASIC	Yes	—	Supported	Supported	Mandatory for all substrates.
CHECKPOINT	Yes	—	Supported	Supported	Atomic checkpoint creation required.

Capability	Required?	Optional?	QIGEM_v1	PostLattice_v2	Notes
RESTORE	Yes	—	Supported	Supported	Idempotent restore required.
HEALTH_PROBE	Yes	—	Supported	Supported	Must return within 500ms.
PHYSICS_TELEMETRY	Yes	—	Supported (qubit metrics)	Supported (lattice metrics)	Required for PhysicsThreatAlert.
QUANTUM_ENTANGLEMENT	—	Yes	Supported	Not supported	QIGEM_v1 specific.
COHERENCE_WINDOW_REPORTING	—	Yes	Supported	Not applicable	Required if QUANTUM_ENTANGLEMENT supported.
LATTICE_PARAMETER_REPORTING	—	Yes	Not applicable	Supported	Required for PostLattice_v2.
ASYNC_EXECUTE	—	Yes	Supported	Supported	Required for high-throughput workloads.
HOT_SWAP	—	Yes	Not supported (v1.7)	Not supported (v1.7)	Planned for v1.8 substrates.
MULTI_TENANT_ISOLATION	—	Yes	Partial (fidelity limits apply)	Supported	Isolation guarantees are substrate-specific.

4.6 MSL Event Types

Event Name	Payload Schema	Trigger Condition	IEB Delivery
MSL_SUBSTRATE_REGISTERED	{ substrate_id, substrate_type, sai_version, timestamp }	Substrate adapter successfully loaded into SAI Adapter Host.	At-least-once
MSL_SUBSTRATE_VALIDATED	{ substrate_id, capability_set, validation_duration_	Capability negotiation completed	At-least-once

Event Name	Payload Schema	Trigger Condition	IEB Delivery
	ms, timestamp }	successfully.	
MSL_SUBSTRATE_ACTIVATED	{ substrate_id, epoch_id, timestamp }	Substrate assigned to an epoch context and entered ACTIVE state.	Exactly-once
MSL_SUBSTRATE_SUSPENDED	{ substrate_id, reason, alert_id, timestamp }	Governance event or PhysicsThreatAlert triggers suspension.	Exactly-once
MSL_SUBSTRATE_DEACTIVATED	{ substrate_id, reason, final_epoch_id, timestamp }	`deactivate()` call completed.	At-least-once
MSL_SUBSTRATE_FAILED	{ substrate_id, error_code, error_detail, timestamp }	Substrate entered FAILED state due to validation failure or runtime error.	Exactly-once
MSL_CAPABILITY_MISMATCH	{ substrate_id, required_capability, declared_capability, timestamp }	Capability negotiator detects a required capability not satisfied.	Exactly-once
MSL_PHYSICS_TELEMETRY	{ substrate_id, telemetry_payload, measurement_timestamp }	Periodic telemetry collection from substrate (polling interval configurable).	At-most-once

4.7 Error Handling and Fallback Semantics

The MSL MUST implement the following error handling behaviors:

- **SAI Adapter Load Failure:** If a substrate adapter fails to load, the MSL MUST publish MSL_SUBSTRATE_FAILED, leave the substrate in FAILED state, and MUST NOT proceed to the VALIDATING lifecycle phase.
- **health_probe() Timeout:** If `health_probe()` does not return within the configured timeout (default: 500ms), the MSL MUST record the timeout, increment the substrate's consecutive-timeout counter, and transition to SUSPENDED state if the counter exceeds the configured threshold (default: 3).
- **execute() Failure:** If `execute()` returns an error or times out, the MSL MUST notify the Epoch Engine via IEB (event: EPOCH_SUBSTRATE_EXEC_FAILED)

and SHOULD attempt to complete the operation on a fallback substrate if one is registered and ACTIVE.

- **Checkpoint Failure:** If `checkpoint()` fails, the MSL MUST NOT acknowledge epoch advancement to the Epoch Engine. The Epoch Engine MUST treat this as a checkpoint invariant violation and initiate rollback to the previous valid checkpoint.
- **Fallback Substrate:** If a primary substrate enters SUSPENDED or FAILED state, the MSL SHOULD activate a designated fallback substrate if declared in the substrate configuration. If no fallback is available, the MSL MUST publish a SUBSTRATE_HALT alert via PhysicsThreatAlert.

4.8 MSL Invariants

The following invariants are formally binding on all conformant MSL implementations. Violation of any invariant constitutes a conformance failure and MUST be reported as a critical audit event.

18. **MSL-INV-001:** At no time SHALL a substrate in state REGISTERED, STANDBY, SUSPENDED, DEACTIVATED, or FAILED accept `execute()` calls. Only ACTIVE substrates MAY accept execution requests.
19. **MSL-INV-002:** Every substrate MUST complete the VALIDATING lifecycle phase successfully before entering STANDBY or ACTIVE. No substrate MAY skip the VALIDATING phase.
20. **MSL-INV-003:** The MSL MUST maintain exactly one SAI adapter instance per registered substrate at all times. Duplicate adapter instances for a single substrate_id are prohibited.
21. **MSL-INV-004:** Physics telemetry MUST be collected from every ACTIVE substrate at least once per configured polling interval. A polling failure MUST be recorded and counted toward the consecutive-timeout threshold.
22. **MSL-INV-005:** A substrate MAY NOT be removed from the Substrate Registry once registered, even after reaching DEACTIVATED or FAILED state. The

registry entry MUST be retained for audit purposes for the full audit retention period defined in §12.5.

23. **MSL-INV-006:** All MSL lifecycle state transitions MUST be published to the IEB and recorded in the audit log before the transition is considered complete.
24. **MSL-INV-007:** The MSL MUST NOT expose substrate-specific adapter internals, hardware identifiers, or raw telemetry data to layers above the MSL except through the formally defined MSL event schema.
25. **MSL-INV-008:** In the event of MSL component failure, the SUBSTRATE_HALT protocol MUST be initiated. The MSL MUST NOT silently degrade or produce partial results after an internal component failure.

---

§5 — Substrate Definitions

5.1 QIGEM_v1 — Quantum-Integrated Generalized Execution Medium, Version 1

5.1.1 Overview and Intended Use Cases

QIGEM_v1 is the first formally defined quantum substrate in the CK platform. It targets gate-model quantum processing units (QPUs) capable of executing parameterized quantum circuits under the governance of the CK MSL. Intended use cases include quantum-enhanced optimization workloads, quantum-assisted cryptographic key generation under MSL oversight, and hybrid classical-quantum cognitive computation pipelines where quantum subroutines are invoked by the Epoch Engine via the SAI.

5.1.2 Physical Characteristics and Constraints

QIGEM_v1 imposes the following constraints on the underlying quantum hardware:

- MUST support gate-model quantum computation (circuit-based, NOT measurement-based or annealing).

- MUST provide qubit count sufficient for the declared workload profile. Minimum 50 physical qubits for any QIGEM_v1 deployment.
- MUST support at minimum: single-qubit gates (X, Y, Z, H, S, T), two-qubit gates (CNOT, CZ), and measurement in the computational basis.
- Classical control hardware MUST be capable of real-time feedback (latency < 1 microsecond from measurement to gate application) for quantum error correction protocols.

5.1.3 Coherence Window Parameters

Parameter	Symbol	Minimum Acceptable	Target (Recommended)	Unit
T1 Relaxation Time	T1	50	≥ 200	microseconds
T2 Dephasing Time	T2	30	≥ 100	microseconds
Circuit Execution Window	T_exec	< T2 / 2	< T2 / 5	microseconds
Coherence Monitoring Interval	T_mon	100	10	milliseconds

If T1 or T2 drops below minimum acceptable values during ACTIVE operation, the MSL MUST trigger a PhysicsThreatAlert of class COHERENCE_COLLAPSE.

5.1.4 Gate Fidelity Requirements

Gate Type	Minimum Average Fidelity	Measurement Method
Single-qubit gate	≥ 99.5%	Randomized benchmarking
Two-qubit gate (CNOT/CZ)	≥ 99.0%	Interleaved randomized benchmarking
State preparation	≥ 99.8%	State tomography (spot-check)
Measurement fidelity	≥ 99.0%	Repeated measurement statistics

5.1.5 Entanglement Topology

Topology Type	Max Qubits (v1.7)	Decoherence Bound	Notes
Linear chain	128	T2 ≥ 30μs	Nearest-neighbor CZ gates only.
2D grid	256	T2 ≥ 50μs	Most common superconducting topology.
Heavy-hex	512	T2 ≥ 80μs	Preferred for error correction codes.
All-to-all (trapped-ion)	64	T2 ≥ 1s (1,000,000μs)	Long coherence; lower gate speed.

5.1.6 Interface Compliance with SAI

The QIGEM_v1 SAI adapter MUST implement all REQUIRED SAI capabilities (§4.5) and MUST additionally implement: QUANTUM_ENTANGLEMENT and COHERENCE_WINDOW_REPORTING. The adapter MUST provide physics telemetry in the QIGEM_TELEMETRY format, which includes at minimum: qubit_count, t1_distribution, t2_distribution, gate_fidelity_map, error_rate_map, and current_circuit_depth.

5.1.7 Known Limitations and Mitigations

Limitation	Impact	Mitigation
Coherence window finite duration	Long circuits may exceed T2, causing decoherence errors.	Epoch Engine MUST bound circuit depth. PhysicsThreatAlert monitors T2.
Gate error accumulation	Error rate grows with circuit depth.	Quantum error correction layers SHOULD be used for circuits > 100 gates.
Measurement back-action	Measurement disturbs unmeasured qubits in some topologies.	MSL adapter MUST enforce measurement scheduling to minimize cross-qubit disturbance.
No HOT_SWAP support in v1.7	QIGEM_v1 cannot be swapped while ACTIVE.	Epoch MUST be FINALIZED before QIGEM_v1 substrate can be replaced. Planned for v1.8.

5.1.8 QIGEM_v1 Configuration Schema

```
{   "substrate_type": "QIGEM_v1",   "substrate_id":
"QIGEM_v1_{instance_uuid}",   "sai_version": "1.7",   "hardware_profile": {
"qubit_count": 127,     "topology": "heavy-hex",     "vendor": "string",
"hw_model": "string",     "firmware_version": "string"   },
"coherence_params": {     "t1_min_us": 50,     "t2_min_us": 30,
"t1_target_us": 200,     "t2_target_us": 100,     "monitoring_interval_ms":
10   },   "gate_fidelity_thresholds": {     "single_qubit_min": 0.995,
"two_qubit_min": 0.990,     "measurement_min": 0.990   },
"physics_alert_config": {     "coherence_collapse_threshold_t2_fraction":
0.5,     "entanglement_poisoning_fidelity_drop_threshold": 0.005,
"alert_polling_interval_ms": 10   },   "fallback_substrate_id":
"PostLattice_v2_{instance_uuid}",   "checkpoint_strategy":
"full_state_snapshot",   "execution_timeout_ms": 30000 }
```

5.2 PostLattice_v2 — Post-Quantum Lattice Substrate, Version 2

5.2.1 Overview and Intended Use Cases

PostLattice_v2 is a post-quantum substrate that operates on classical hardware but employs lattice-based cryptographic primitives as the computational foundation for tamper-evident, quantum-resistant operation. It is intended for deployments where quantum hardware is unavailable, prohibitively expensive, or insufficiently reliable, but post-quantum security guarantees are still required. Use cases include secure multi-party epoch consensus, quantum-resistant audit log generation, and post-quantum cryptographic key lifecycle management within the CK governance framework.

5.2.2 Lattice Parameters

Parameter	Value (v1.7 Default)	Allowed Range	Notes
Dimension (n)	1024	512–4096	Higher n = stronger security, higher cost.
Modulus (q)	12289	Prime, q ≡ 1 (mod 2n)	Enables NTT-based fast polynomial multiplication.
Error Distribution	Discrete Gaussian σ=3.2	σ ∈ [2.0, 5.0]	Follows NIST SP 800-208 recommendations.
Security Level (NIST)	Level 3 (equivalent AES-192)	Level 1–5	Configurable via dimension/modulus selection.

Parameter	Value (v1.7 Default)	Allowed Range	Notes
Polynomial Ring	Z_q[x] / (x^n + 1)	—	Standard Ring-LWE ring structure.

5.2.3 Hardness Assumptions

PostLattice_v2 security is grounded in the following computational hardness assumptions, consistent with the NIST Post-Quantum Cryptography standardization process (FIPS 203, 204):

- **Learning With Errors (LWE):** It is computationally infeasible, under quantum adversary models, to distinguish (A, As + e) from uniform random, where A is a public matrix, s is a secret vector, and e is a small-norm error vector drawn from the configured Gaussian distribution.
- **Ring-LWE:** The ring-structured variant of LWE, operating in Z_q[x]/(x^n + 1), providing equivalent security with smaller key sizes and faster operations via NTT.
- **Module-LWE:** The module-structured variant used for the consensus signature scheme within PostLattice_v2, providing a trade-off between Ring-LWE and standard LWE.

5.2.4 Throughput and Latency Profile

Operation	Throughput (ops/sec)	Latency (p50)	Latency (p99)	Hardware Baseline
Key Generation	5,000	0.2ms	0.5ms	Single modern CPU core
Encapsulation (KEM)	12,000	0.08ms	0.2ms	Single modern CPU core
Decapsulation (KEM)	11,000	0.09ms	0.22ms	Single modern CPU core
Signature Generation	3,500	0.28ms	0.7ms	Single modern CPU core
Signature Verification	8,000	0.12ms	0.3ms	Single modern CPU core
Checkpoint (1MB epoch	200	5ms	12ms	NVMe SSD backing store

Operation	Throughput (ops/sec)	Latency (p50)	Latency (p99)	Hardware Baseline
state)				

5.2.5 SAI Compliance

The PostLattice_v2 SAI adapter MUST implement all REQUIRED SAI capabilities and MUST additionally implement LATTICE_PARAMETER_REPORTING. Physics telemetry for PostLattice_v2 covers: current_lattice_dimension, current_modulus, error_distribution_sigma, key_generation_error_rate, and ntt_performance_deviation_percent.

5.2.6 Upgrade Path from PostLattice_v1

PostLattice_v1 is deprecated in CK v1.7. The following upgrade steps MUST be followed:

26. Register the PostLattice_v2 adapter alongside the existing PostLattice_v1 adapter (both may coexist in STANDBY simultaneously during migration).
27. Execute the CK migration toolchain command: `ck-migrate substrate upgrade --from PostLattice_v1 --to PostLattice_v2 --verify`
28. Finalize the current epoch on PostLattice_v1 before activating PostLattice_v2.
29. Validate PostLattice_v2 capability matrix compliance using the post-migration validation suite (§11.7).
30. Deactivate PostLattice_v1 via MSL. Do NOT remove its registry entry (audit retention required).

5.2.7 PostLattice_v2 Configuration Schema

```
{   "substrate_type": "PostLattice_v2",   "substrate_id":
"PostLattice_v2_{instance_uuid}",   "sai_version": "1.7",   "lattice_params":
{     "dimension_n": 1024,     "modulus_q": 12289,     "error_distribution":
"discrete_gaussian",     "error_sigma": 3.2,     "security_level_nist": 3,
"ring_structure": "Z_q[x]/(x^n+1)"   },   "performance_config":
{     "ntt_acceleration": true,     "avx512_enabled": true,
"thread_pool_size": 8   },   "physics_alert_config":
{     "lattice_parameter_drift_sigma_threshold": 0.05,
"ntt_performance_deviation_alert_percent": 10,
"alert_polling_interval_ms": 100   },   "checkpoint_strategy":
"incremental_delta",   "execution_timeout_ms": 5000,
"fallback_substrate_id": null }
```

5.3 Substrate Selection Policy

5.3.1 Decision Tree for Substrate Selection

When configuring a CK v1.7 deployment, the substrate selection SHOULD follow the decision logic below:

31. Does the workload require quantum superposition or entanglement primitives? → Select QIGEM_v1 (if hardware available and meets coherence requirements).
32. Is post-quantum cryptographic security required without quantum hardware? → Select PostLattice_v2.
33. Are both quantum capability and post-quantum security required? → Configure QIGEM_v1 as primary with PostLattice_v2 as fallback.
34. Does available quantum hardware satisfy QIGEM_v1 fidelity thresholds (§5.1.4)? If not → Fall back to PostLattice_v2 as sole substrate.
35. If no substrate satisfies all capability requirements → Halt deployment. MUST NOT proceed with an uncapable substrate.

5.3.2 Multi-Substrate Configurations

CK v1.7 supports multi-substrate configurations where multiple substrates are registered simultaneously. In such configurations:

- Each substrate MUST be assigned a distinct role: PRIMARY, FALLBACK, or ANCILLARY.
- Exactly one substrate MUST hold the PRIMARY role at any given time.
- At most one substrate MAY hold the FALLBACK role per PRIMARY substrate.
- ANCILLARY substrates MAY receive execution requests concurrently with the PRIMARY substrate, for independent workloads only.

5.3.3 Substrate Failover Rules

Failover from PRIMARY to FALLBACK substrate is triggered by: (a) PRIMARY substrate entering SUSPENDED or FAILED state; (b) MSL receiving a SUBSTRATE_HALT alert for the PRIMARY substrate. Upon failover trigger:

36. The Epoch Engine is notified via IEB event EPOCH_SUBSTRATE_FAILOVER_INITIATED.
37. The current epoch is checkpointed on the PRIMARY substrate if still possible; otherwise the last valid checkpoint is used.
38. The FALLBACK substrate is promoted to PRIMARY.
39. The checkpoint is restored on the new PRIMARY substrate.
40. The Epoch Engine resumes from the restored checkpoint. A governance audit event is published.

---

§6 — Epoch Engine (v1.7 Substrate-Agnostic Design)

6.1 Epoch Definition and Purpose

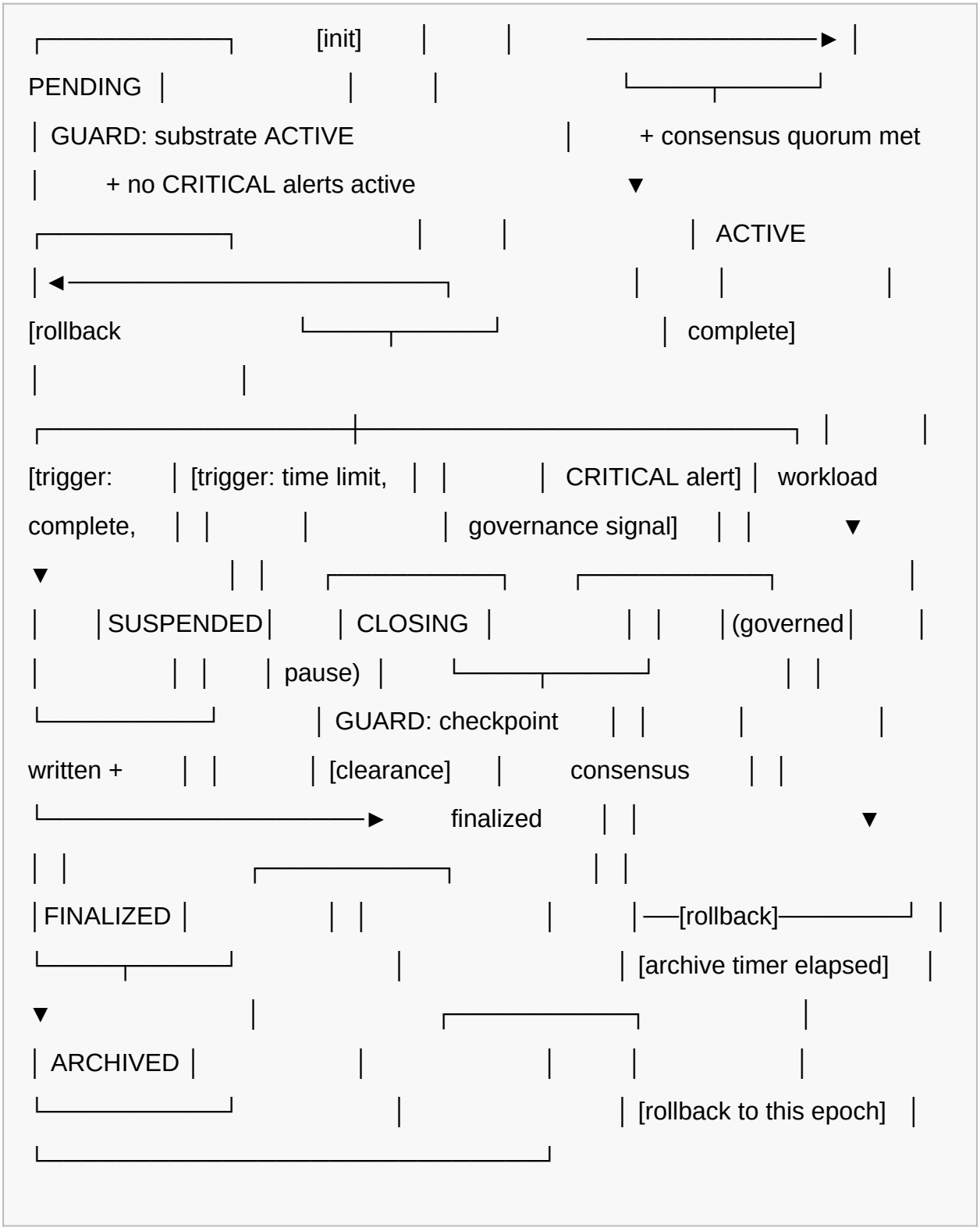
An **epoch** in CK v1.7 is a formally bounded unit of computation: a discrete, checkpointed, auditable segment of the CK runtime's operational history. Each epoch has a unique identifier, a defined start and end boundary, an associated substrate context, a consensus state snapshot, and a complete audit record. Epochs are the fundamental unit of governance, rollback, and compliance in CK v1.7.

Epochs serve the following functions:

- **Isolation:** Each epoch provides a bounded computational context, preventing state leakage between discrete operational periods.
- **Auditability:** Epoch boundaries define the granularity of the audit log and compliance records.
- **Rollback:** In the event of a governance violation or physics-threat event, the CK runtime may roll back to the beginning of any epoch for which a valid checkpoint exists.
- **Substrate Abstraction:** Epoch interfaces are substrate-agnostic; the Epoch Engine interacts with substrates only via MSL/SAI.

6.2 Epoch Lifecycle State Machine

Each epoch passes through the following states. All transitions are deterministic and governed by formal guards.



Transition	From State	To State	Guard Conditions	Action
INIT	—	PENDING	MSL has at least one ACTIVE substrate; IEB operational.	Assign epoch_id, record start timestamp.
ACTIVATE	PENDING	ACTIVE	Substrate ACTIVE; consensus quorum met; no CRITICAL/SUBSTRATE_HALT alerts.	Publish EPOCH_ACTIVATED event.
CLOSE	ACTIVE	CLOSING	Closing trigger received (time limit, workload complete, governance signal).	Freeze new workload admission; begin checkpoint.
FINALIZE	CLOSING	FINALIZED	Checkpoint written and verified; consensus contract finalized for epoch.	Publish EPOCH_FINALIZED; write final audit record.
ARCHIVE	FINALIZED	ARCHIVED	Archive timer elapsed (configurable, default: 7 days post-finalization).	Move checkpoint to long-term storage; retain audit record.
SUSPEND	ACTIVE	SUSPENDED	CRITICAL PhysicsThreatAlert or governance SUSPEND signal received.	Pause workload; await governance clearance.
RESUME	SUSPENDED	ACTIVE	Governance clearance signal received; substrate status OK.	Resume workload admission.
ROLLBACK	FINALIZED or ARCHIVED	ACTIVE (restored)	Rollback authorization received; valid checkpoint available.	Restore checkpoint on current PRIMARY substrate; resume from

Transition	From State	To State	Guard Conditions	Action
				restored state.

6.3 Substrate-Agnostic Epoch Interface

The Epoch Engine communicates with substrates exclusively via the MSL. The Epoch Engine MUST NOT contain any substrate-type-specific logic. The substrate-agnostic epoch interface is expressed as the following IEB events emitted by the Epoch Engine to the MSL:

- `EPOCH_EXEC_DISPATCH` — Dispatch a substrate operation for the current epoch.
- `EPOCH_CHECKPOINT_REQUEST` — Request a substrate checkpoint for the current epoch_id.
- `EPOCH_RESTORE_REQUEST` — Request substrate state restoration from a checkpoint handle.
- `EPOCH_DEACTIVATE_SUBSTRATE` — Signal MSL to begin substrate deactivation after epoch finalization.

6.4 Epoch Trigger Conditions

Trigger Type	Condition	Priority	Action
TIME_LIMIT	Epoch wall-clock duration exceeds configured max_epoch_duration.	Normal	Initiate CLOSING transition.
WORKLOAD_COMPLETE	All admitted workloads in the epoch have returned results.	Normal	Initiate CLOSING transition.
GOVERNANCE_SIGNAL	Consensus Layer issues EPOCH_CLOSE_SIGNAL via IEB.	High	Initiate CLOSING transition immediately.
PHYSICS_THREAT_CRITICAL	PhysicsThreatAlert publishes CRITICAL alert for the active substrate.	Critical	Initiate SUSPEND transition; pause all workloads.

Trigger Type	Condition	Priority	Action
SUBSTRATE_HALT_ALERT	PhysicsThreatAlert publishes SUBSTRATE_HALT alert.	Emergency	Initiate SUSPEND transition; escalate to human-in-the-loop.
CONSENSUS_QUORUM_LOST	Consensus Contract reports quorum below minimum threshold.	High	Initiate SUSPEND transition; await quorum recovery.
CHECKPOINT_FAILURE	MSL reports checkpoint failure for the current epoch.	Critical	Halt CLOSING; rollback to last valid checkpoint.
MANUAL_TRIGGER	Authorized operator issues manual epoch boundary command.	Normal	Initiate CLOSING transition after authorization verification.

6.5 Epoch Transition Protocol

When a CLOSING trigger is received, the Epoch Engine MUST execute the following transition protocol in order:

41. **Admission Freeze:** Cease accepting new workload submissions for the current epoch.
42. **Workload Drain:** Wait for all in-flight workloads to complete or time out. Timeout is configurable (default: 30 seconds). Timed-out workloads are recorded as INCOMPLETE in the epoch audit record.
43. **State Snapshot:** Request a complete epoch state snapshot from all ACTIVE substrates via EPOCH_CHECKPOINT_REQUEST.
44. **Checkpoint Verification:** Verify that all checkpoint handles returned by MSL are valid (non-null, hash-verified).
45. **Consensus Finalization:** Submit the epoch state hash to the Consensus Contract for finalization. Await consensus quorum acknowledgment.
46. **Audit Record Flush:** Write the complete epoch audit record (workload count, state hash, checkpoint handle, timestamp, substrate_id) to the audit log.

47. **FINALIZED Transition:** Publish EPOCH_FINALIZED event on IEB. Epoch state machine transitions to FINALIZED.
48. **Next Epoch Init:** Initialize the next epoch in PENDING state.

6.6 Epoch Rollback Semantics

CK v1.7 guarantees deterministic epoch rollback for any epoch that has a valid checkpoint. Rollback is the sole mechanism for recovering from confirmed state corruption, governance violations, or physics-threat-induced state invalidation.

Rollback eligibility: An epoch is eligible for rollback if and only if: (a) a valid, verified checkpoint exists for that epoch, (b) the rollback has been authorized by the Consensus Contract (quorum required), and (c) the target epoch's state hash is consistent with the checkpoint handle.

Rollback MUST be idempotent: initiating rollback to the same checkpoint twice MUST produce identical outcomes.

The rolled-back epoch MUST be re-entered in ACTIVE state from the restored checkpoint, not in PENDING state. All workloads that were in-flight at rollback initiation are considered VOIDED and MUST be re-submitted.

6.7 Epoch Checkpointing

Checkpointing is performed by the MSL via the SAI `checkpoint(epoch_id)` call. The Epoch Engine MUST request checkpoints at the following events:

- At every CLOSING trigger (mandatory; epoch MUST NOT advance to FINALIZED without a verified checkpoint).
- At configurable intra-epoch intervals (default: every 5 minutes within a long-running epoch).
- Upon receipt of a PHYSICS_THREAT_WARNING alert (proactive checkpoint before potential escalation).

Checkpoints are identified by a CHECKPOINT_HANDLE containing: epoch_id, sequence_number, state_hash (SHA-256), substrate_id, and creation_timestamp_utc.

6.8 Epoch Engine Invariants

49. **EE-INV-001:** At most one epoch SHALL be in ACTIVE or CLOSING state at any given time per CK runtime instance.
50. **EE-INV-002:** No epoch SHALL transition from PENDING to ACTIVE unless the MSL reports at least one substrate in ACTIVE state.
51. **EE-INV-003:** No epoch SHALL transition to FINALIZED without a verified, non-null checkpoint handle.
52. **EE-INV-004:** Epoch identifiers MUST be globally unique, monotonically increasing, and tamper-evident. The epoch_id format is: `{runtime_id}:{epoch_sequence_uint64}:{creation_timestamp_utc}`.
53. **EE-INV-005:** An epoch in ARCHIVED state MAY be rolled back. An epoch that has been rolled back MUST be re-archived after the rollback is resolved.
54. **EE-INV-006:** Epoch transition events MUST be published to the IEB before the transition is recorded in the local state machine. Events MUST NOT be suppressed.
55. **EE-INV-007:** The Epoch Engine MUST NOT contain any reference to a specific substrate type. All substrate interactions MUST flow through the MSL interface.
56. **EE-INV-008:** In a SUSPENDED epoch, no substrate execute() calls SHALL be dispatched by the Epoch Engine.

6.9 Performance Targets

Metric	v1.6 Baseline	v1.7 Target	Unit
Epoch transition latency (ACTIVE→FINALIZED)	12,000	≤ 5,000	milliseconds
Checkpoint creation time (1MB epoch state)	800	≤ 300	milliseconds
Rollback execution time (from	Not formally defined	≤ 10,000	milliseconds

Metric	v1.6 Baseline	v1.7 Target	Unit
checkpoint)			
Workload drain timeout (p99)	60,000	≤ 30,000	milliseconds
PENDING → ACTIVE transition time	5,000	≤ 2,000	milliseconds
Audit record flush time	2,000	≤ 500	milliseconds
Max epochs per hour (sustained throughput)	120	≥ 360	epochs/hour
Epoch state overhead (per epoch)	~8MB	≤ 4MB	megabytes

---

§7 — Physics-Threat Alert System (PhysicsThreatAlert)

7.1 Motivation — Why Physics-Level Threat Detection

In CK v1.6 and earlier, substrate-level physical anomalies — such as decoherence events in quantum substrates or lattice parameter drift in post-quantum substrates — had no formal detection or response pathway. When such events occurred, the CK runtime experienced undefined behavior, ranging from incorrect computation results to total substrate failure, with no governance record and no rollback opportunity.

CK v1.7 introduces the PhysicsThreatAlert system to formalize the detection, classification, routing, and response to physics-level substrate threats. The system is predicated on the principle that *physical substrate behavior constitutes a governance concern*, not merely an operational concern, because it directly affects the integrity of cognitive computation outputs and the auditability of governance decisions made during affected epochs.

7.2 Threat Taxonomy

Threat Class	Description	Detection Method	Severity Level	Response Action
COHERENCE_COLLAPSE	T1 or T2 coherence times drop below minimum acceptable threshold during active quantum substrate operation. Causes quantum state decoherence and unreliable gate outcomes.	Continuous T1/T2 monitoring via QIGEM_v1 physics telemetry. Alert triggered when T2 < T2_min or T1 < T1_min.	CRITICAL	Suspend epoch; checkpoint current state; initiate substrate failover if T2 < 50% of minimum.
ENTANGLEMENT_POISONING	Anomalous reduction in two-qubit gate fidelity below threshold, indicating entanglement channel degradation, possibly due to environmental noise or cross-qubit leakage.	Gate fidelity map comparison against baseline. Alert triggered when fidelity drop > 0.5% from last calibration.	WARNING or CRITICAL (threshold-dependent)	WARNING: Log and continue with increased error correction. CRITICAL: Suspend epoch; recalibrate.
SUBSTRATE_TIMING_VIOLATION	A substrate operation exceeds the configured execution timeout or violates a defined timing constraint. In quantum substrates, timing violations may exceed the coherence window.	SAI adapter execution timer. Alert triggered when execute() latency exceeds timeout_ms or circuit depth exceeds coherence window budget.	WARNING or CRITICAL	WARNING: Record violation. CRITICAL: Abort operation; rollback to last checkpoint.
LATTICE_PARAMETER_DRIFT	PostLattice_v2 substrate lattice parameters (error distribution sigma, NTT	Continuous monitoring of LATTICE_PARAMETER_REPORTING telemetry. Alert triggered	ADVISORY or WARNING	ADVISORY: Log drift. WARNING: Alert operators; consider parameter reconfiguration.

Threat Class	Description	Detection Method	Severity Level	Response Action
	performance) deviate from configured baseline values. May indicate hardware instability or adversarial lattice manipulation.	when sigma deviation > configured threshold.		
CROSS_SUBSTRATE_INTERFERENCE	In multi-substrate configurations, anomalous correlation between telemetry signals from distinct substrates, suggesting electromagnetic interference, shared resource contention, or an active adversarial cross-substrate attack.	Correlation analysis of simultaneous telemetry from multiple substrates. Alert triggered when cross-substrate correlation coefficient exceeds threshold.	CRITICAL or SUBSTRATE_HALT	CRITICAL: Suspend affected substrates. SUBSTRATE_HALT: Halt all substrates; mandatory human-in-the-loop escalation.

7.3 Alert Pipeline Architecture

The PhysicsThreatAlert pipeline is an internal component of the MSL. Its processing stages are:

57. **Telemetry Ingestion:** Receives physics telemetry from all ACTIVE substrate SAI adapters on the configured polling interval.
58. **Threat Evaluation Engine:** Compares ingested telemetry against the threat taxonomy thresholds. Implemented as a set of formally specified evaluation rules, one per threat class.
59. **Alert Generation:** If a threat condition is detected, an alert record is generated with a unique alert_id, threat_class, severity, and recommended_action.

60. **Severity Classification:** The alert is assigned a severity level (§7.5).
61. **IEB Publication:** The alert is published to the IEB on the PhysicsThreatAlert priority channel, which pre-empts the normal event queue.
62. **Audit Recording:** All alerts, regardless of severity, are written to the audit log before IEB publication.
63. **Response Orchestration:** The recommended_action is dispatched to the appropriate subsystem (Epoch Engine, MSL, Governance escalation handler).

7.4 Alert Schema

Field	Type	Required	Description	Constraints
alert_id	STRING (UUID v4)	Yes	Globally unique alert identifier.	MUST be UUID v4 format. Immutable after generation.
threat_class	ENUM	Yes	Threat classification from the taxonomy (§7.2).	One of: COHERENCE_COLLAPSE, ENTANGLEMENT_POISONING, SUBSTRATE_TIMING_VIOLATION, LATTICE_PARAMETER_DRIFT, CROSS_SUBSTRATE_INTERFERENCE.
substrate_id	STRING	Yes	Identifier of the substrate generating the threat signal.	MUST match a substrate_id in the Substrate Registry.
epoch_id	STRING	Yes	Identifier of the epoch active at alert generation time.	MUST match the current ACTIVE or CLOSING epoch_id.
severity	ENUM	Yes	Alert severity level (§7.5).	One of: ADVISORY, WARNING, CRITICAL, SUBSTRATE_HALT.
timestamp	DATETIME (UTC)	Yes	UTC timestamp of alert generation (not	ISO 8601 format. Millisecond

Field	Type	Required	Description	Constraints
			telemetry acquisition).	precision required.
telemetry_snapshot	OBJECT	Yes	The exact telemetry values that triggered the alert, captured at detection time.	Substrate-type-specific schema. MUST be immutable after capture.
threshold_reference	OBJECT	Yes	The configured threshold values against which the telemetry was evaluated.	Immutable reference to configuration in effect at detection time.
recommended_action	STRING	Yes	Human-readable and machine-parseable recommended response action.	MUST be one of the defined action identifiers in the threat taxonomy.
auto_response_taken	BOOLEAN	Yes	Indicates whether an automated response action was taken.	If true, auto_response_detail MUST be populated.
auto_response_detail	STRING	Conditional	Description of the automated response action executed.	Required if auto_response_taken is true.
human_escalation_required	BOOLEAN	Yes	Indicates whether human-in-the-loop escalation is required (§7.7).	MUST be true for SUBSTRATE_HALT severity.
correlation_id	STRING (UUID v4)	No	Groups related alerts from the same physical event.	Optional; used for cross-substrate interference correlation.

7.5 Alert Severity Levels

Severity	Description	Automated Response	Human Escalation	Epoch Impact
ADVISORY	Informational. Telemetry	Log alert; increase	Not required.	None. Epoch continues

Severity	Description	Automated Response	Human Escalation	Epoch Impact
	approaching threshold but not yet violated. Monitor situation.	telemetry polling frequency.		normally.
WARNING	Threshold mildly violated. Situation requires attention but does not immediately threaten epoch integrity.	Log alert; notify application layer; request proactive checkpoint.	Notification sent; response optional.	Intra-epoch checkpoint triggered. No state suspension.
CRITICAL	Threshold significantly violated. Epoch integrity at risk. Immediate response required.	Suspend epoch; initiate substrate failover evaluation; log alert with full telemetry snapshot.	Required. Escalation to designated governance officer. 15-minute response SLA.	Epoch SUSPENDED. Must receive governance clearance to resume.
SUBSTRATE_HALT	Catastrophic substrate condition. All substrate operations MUST cease immediately. State may be corrupted.	Halt all substrate execute() calls; attempt emergency checkpoint if substrate still responsive; publish SUBSTRATE_HALT event on IEB priority channel.	Mandatory. Immediate escalation. Runtime MUST NOT resume without explicit human authorization.	Epoch SUSPENDED. Full runtime enters SUBSTRATE_HALT lifecycle phase. Resume requires human authorization and post-incident review.

7.6 Alert Routing via Internal Event Bus

PhysicsThreatAlert events are published to the IEB on the **PhysicsThreatAlert priority channel**, which is a dedicated IEB channel separate from normal event queues. The priority channel MUST pre-empt all normal channel processing. All IEB consumers MUST register a PhysicsThreatAlert handler if they are affected by substrate conditions. Specifically:

- The **Epoch Engine** MUST subscribe to all PhysicsThreatAlert events and respond per the trigger condition table (§6.4).

- The **Consensus Layer** MUST subscribe to CRITICAL and SUBSTRATE_HALT events and suspend consensus operations accordingly.
- The **Application Layer** MUST receive ADVISORY and above events and SHOULD propagate them to consuming applications.

7.7 Human-in-the-Loop Escalation Policy

The following escalation rules govern human-in-the-loop response to PhysicsThreatAlert events:

- **ADVISORY:** No escalation required. Alerts are available in the observability dashboard and audit log.
- **WARNING:** Escalation notification MUST be sent to the registered governance notification endpoint. Response is optional within 1 hour. Automated systems continue operation.
- **CRITICAL:** Escalation notification MUST be sent within 60 seconds of alert generation. The designated governance officer MUST acknowledge receipt within 15 minutes. The CK runtime remains in GOVERNED_PAUSE until acknowledgment. After acknowledgment, the officer MAY authorize RESUME or ROLLBACK.
- **SUBSTRATE_HALT:** All escalation contacts MUST be notified immediately. The CK runtime enters SUBSTRATE_HALT lifecycle phase and MUST NOT resume without explicit written authorization (in the governance system) from an authorized officer. A post-incident review document MUST be submitted to the audit record within 72 hours.

7.8 PhysicsThreatAlert Invariants

64. **PTA-INV-001:** Every physics telemetry collection cycle MUST result in a threat evaluation pass. Skipped evaluation cycles are not permitted.
65. **PTA-INV-002:** All alerts, regardless of severity, MUST be written to the audit log before any automated response action is taken.

66. **PTA-INV-003:** SUBSTRATE_HALT alerts MUST result in the suspension of ALL substrate execute() calls within 100ms of alert generation, regardless of in-flight operations.
67. **PTA-INV-004:** Alert records MUST be immutable after generation. No post-generation modification of alert fields is permitted.
68. **PTA-INV-005:** The PhysicsThreatAlert Engine MUST itself be monitored for liveness. If the engine becomes unresponsive, the MSL MUST treat this as a SUBSTRATE_HALT condition.
69. **PTA-INV-006:** human_escalation_required MUST be true for every SUBSTRATE_HALT alert. No exception or override is permitted.

7.9 Example Alert Payloads

Example 1: COHERENCE_COLLAPSE (CRITICAL)


```
{   "alert_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",   "threat_class":
"COHERENCE_COLLAPSE",   "substrate_id": "QIGEM_v1_a3f7c291",   "epoch_id":
"ck-runtime-01:00000047:2026-06-15T02:14:33.117Z",   "severity": "CRITICAL",
"timestamp": "2026-06-15T02:14:33.891Z",   "telemetry_snapshot":
{     "t1_us": 48.2,     "t2_us": 22.7,     "qubit_count": 127,
"topology": "heavy-hex"   },   "threshold_reference": {     "t1_min_us": 50,
"t2_min_us": 30   },   "recommended_action":
"SUSPEND_EPOCH_INITIATE_FAILOVER",   "auto_response_taken": true,
"auto_response_detail": "Epoch suspended. Failover to PostLattice_v2_bb9e1234
initiated.",   "human_escalation_required": true,   "correlation_id": null }
```


Example 2: LATTICE_PARAMETER_DRIFT (ADVISORY)


```
{   "alert_id": "3d6f3c4a-1092-4c9a-b7f2-0a44de8c9012",   "threat_class":
"LATTICE_PARAMETER_DRIFT",   "substrate_id": "PostLattice_v2_bb9e1234",
"epoch_id": "ck-runtime-01:00000048:2026-06-15T02:15:01.003Z",   "severity":
"ADVISORY",   "timestamp": "2026-06-15T02:15:01.444Z",
"telemetry_snapshot": {     "error_sigma_current": 3.37,
"error_sigma_baseline": 3.2,     "ntt_performance_deviation_percent":
1.2   },   "threshold_reference":
{     "lattice_parameter_drift_sigma_threshold": 0.05,
"ntt_performance_deviation_alert_percent": 10   },   "recommended_action":
"LOG_AND_INCREASE_TELEMETRY_POLLING",   "auto_response_taken": true,
"auto_response_detail": "Telemetry polling interval decreased to 10ms. Alert
logged.",   "human_escalation_required": false,   "correlation_id": null }
```


Example 3: CROSS_SUBSTRATE_INTERFERENCE (SUBSTRATE_HALT)


```
{   "alert_id": "9a1b2c3d-4e5f-6789-abcd-ef0123456789",   "threat_class":
"CROSS_SUBSTRATE_INTERFERENCE",   "substrate_id": "QIGEM_v1_a3f7c291",
"epoch_id": "ck-runtime-01:00000049:2026-06-15T02:16:44.221Z",   "severity":
"SUBSTRATE_HALT",   "timestamp": "2026-06-15T02:16:44.887Z",
"telemetry_snapshot": {     "cross_correlation_coefficient": 0.94,
"interfering_substrate_id": "PostLattice_v2_bb9e1234",
```

```
"interference_frequency_hz": 432.1,     "affected_qubit_indices": [12, 13,
14, 15, 16]   },   "threshold_reference":
{     "cross_substrate_correlation_halt_threshold": 0.9   },
"recommended_action": "HALT_ALL_SUBSTRATES_MANDATORY_HUMAN_REVIEW",
"auto_response_taken": true,   "auto_response_detail": "All substrate
execute() calls halted. Emergency checkpoint attempted. Human escalation
sent.",   "human_escalation_required": true,   "correlation_id": "7f8e9d0c-
1b2a-3c4d-5e6f-7a8b9c0d1e2f" }
```


Example 4: ENTANGLEMENT_POISONING (WARNING)


```
{   "alert_id": "c0ffee01-dead-beef-cafe-123456789abc",   "threat_class":
"ENTANGLEMENT_POISONING",   "substrate_id": "QIGEM_v1_a3f7c291",
"epoch_id": "ck-runtime-01:00000050:2026-06-15T02:17:55.009Z",   "severity":
"WARNING",   "timestamp": "2026-06-15T02:17:55.342Z",   "telemetry_snapshot":
{     "two_qubit_gate_fidelity_current": 0.9843,
"two_qubit_gate_fidelity_baseline": 0.9901,     "fidelity_drop": 0.0058,
"worst_affected_pair": [42, 43]   },   "threshold_reference":
{     "entanglement_poisoning_fidelity_drop_threshold": 0.005   },
"recommended_action": "REQUEST_PROACTIVE_CHECKPOINT_NOTIFY_OPERATORS",
"auto_response_taken": true,   "auto_response_detail": "Proactive checkpoint
requested for epoch. Operator notification dispatched.",
"human_escalation_required": false,   "correlation_id": null }
```


---

§8 — Consensus Contract v1.7

8.1 Overview and Purpose

The Consensus Contract is the formal agreement governing distributed state agreement among CK runtime instances operating in a multi-node deployment. In CK v1.7, the Consensus Contract has been comprehensively reformulated from the informal v1.6.3 guidelines into a formally specified, machine-verifiable contract with numbered clauses, formal invariants, and machine-enforceable quorum rules.

8.2 Contract Parties and Roles

Role	Description	Minimum Count	Maximum Count
PROPOSER	A CK runtime instance authorized to propose epoch state transitions and consensus decisions. MUST hold a valid governance credential.	1	Unbounded

Role	Description	Minimum Count	Maximum Count
VALIDATOR	A CK runtime instance authorized to validate and vote on proposals. MUST independently verify all proposed state transitions before voting.	3	Unbounded
OBSERVER	A CK runtime instance that receives consensus outcomes but does not participate in voting. May produce audit records.	0	Unbounded
ARBITER	A designated CK runtime instance or external governance authority that resolves tie votes and conflict conditions. MUST be pre-agreed in the contract configuration.	0	1

8.3 v1.7 Contract Terms

70. **Clause 1 — Proposal Validity:** A proposal MUST include: the proposing instance's identity, the proposed epoch state hash, the previous epoch state hash, the substrate_id of the proposing instance's active substrate, and a valid cryptographic signature using the proposer's governance credential key.
71. **Clause 2 — Independent Verification:** Each VALIDATOR MUST independently verify the proposed state transition by comparing the proposed state hash against its own local computation of the expected state hash. Validators MUST NOT coordinate vote decisions before independent verification is complete.
72. **Clause 3 — Vote Casting:** Each VALIDATOR MUST cast exactly one vote per proposal: ACCEPT, REJECT, or ABSTAIN. ABSTAIN is permitted only in documented exceptional circumstances (network partition, health probe

failure). Multiple votes for the same proposal by the same validator are not permitted.

73. **Clause 4 — Quorum Requirement:** A proposal is accepted if and only if: (ACCEPT_count / (ACCEPT_count + REJECT_count)) ≥ quorum_threshold AND ACCEPT_count ≥ minimum_accept_count. Default quorum_threshold: 2/3. Default minimum_accept_count: 2.
74. **Clause 5 — Proposal Timeout:** A proposal that does not achieve quorum or a clear rejection within the proposal_timeout period (default: 60 seconds) is considered LAPSED. A lapsed proposal MUST be re-submitted or the epoch transition MUST be aborted.
75. **Clause 6 — Epoch State Binding:** Upon acceptance, the proposed epoch state hash is binding on all VALIDATOR and OBSERVER instances. All instances MUST converge to the accepted state within the convergence_timeout period (default: 30 seconds after quorum).
76. **Clause 7 — Non-Repudiation:** All votes MUST be signed with the voting instance's governance credential key. Unsigned votes MUST be rejected. Signed votes are non-repudiable and are retained in the audit log.
77. **Clause 8 — Audit Log Completeness:** Every proposal, every vote, every quorum result, and every conflict resolution action MUST be recorded in the audit log within 1 second of occurrence.
78. **Clause 9 — Governance Credential Rotation:** Governance credentials MUST be rotated at minimum every 90 days. A credential rotation MUST be proposed and accepted via the consensus mechanism itself.
79. **Clause 10 — Contract Immutability:** The Consensus Contract terms for a given CK version are immutable. Modification of contract terms requires a major version upgrade of the CK specification.

8.4 Formal Invariants

The following formal invariants are expressed in plain-text predicate notation. All conformant implementations MUST enforce these invariants at runtime.

80. **CC-INV-001 (Agreement):** For all accepted proposals P: if instance_i accepts P and instance_j accepts P, then state_hash(instance_i) = state_hash(instance_j) after convergence. No two validators SHALL accept proposals with conflicting state hashes for the same epoch.
81. **CC-INV-002 (Validity):** For all accepted proposals P: proposal_state_hash(P) MUST be a valid state transition from the immediately preceding accepted state hash. Proposals whose state hash is not a valid transition from the prior accepted state MUST be REJECTED.
82. **CC-INV-003 (Termination):** Every proposal MUST eventually be either ACCEPTED, REJECTED, or LAPSED within the proposal_timeout. The consensus mechanism MUST NOT deadlock.
83. **CC-INV-004 (Non-Duplication):** No epoch state hash SHALL appear as the accepted hash of more than one distinct epoch. Epoch identifiers and state hashes MUST be globally unique.
84. **CC-INV-005 (Liveness):** In the absence of Byzantine faults exceeding the fault tolerance threshold (1/3 of validators), the consensus mechanism MUST reach a decision on every proposal within 2 × proposal_timeout.

8.5 Consensus Quorum Rules

Scenario	Quorum Threshold	Minimum ACCEPT Count	Arbiter Invoked?
Normal operation (3+ validators)	2/3	2	No
Degraded operation (exactly 2 validators available)	1/1 (unanimous)	2	No (unanimous required)
Tie vote with Arbiter configured	Arbiter casts deciding vote	Arbiter ACCEPT or majority	Yes
Tie vote without Arbiter configured	Proposal LAPSED	N/A	N/A (lapsed)
Physics-threat CRITICAL during vote	Vote suspended; epoch SUSPENDED	N/A (pending clearance)	Pending

8.6 Conflict Resolution Protocol

When a conflict is detected (two PROPOSERS submitting conflicting proposals for the same epoch), the following resolution protocol applies:

85. Both proposals are suspended and marked CONFLICTED.
86. All validators publish their local state hash to the audit log.
87. The Arbiter (if present) reviews both proposals and designates one as authoritative, or requests re-proposal.
88. If no Arbiter is present, the proposal with the highest validator ACCEPT count at time of conflict detection is preferred; if equal, the proposal with the earlier creation_timestamp is preferred.
89. The non-authoritative proposal is marked REJECTED_CONFLICT. Its proposer MUST re-submit from the authoritative state.
90. All conflict resolution actions are recorded in the audit log.

8.7 Contract Violation Handling

When a contract violation is detected (e.g., a validator casting multiple votes, a proposal without a valid signature), the following actions MUST be taken:

- The violating action MUST be rejected and recorded in the audit log as a CONTRACT_VIOLATION event.
- If the violation is attributable to a specific runtime instance, that instance's voting rights MUST be suspended for the current epoch.
- A PhysicsThreatAlert of severity WARNING or CRITICAL (depending on violation severity) MUST be emitted if the violation threatens epoch integrity.
- Repeated violations by a single instance MUST trigger a governance escalation event.

8.8 Audit Log Requirements

The Consensus Layer MUST write the following records to the audit log:

- Every proposal submitted (proposal_id, proposer_id, state_hash, timestamp, signature).
- Every vote cast (vote_id, proposal_id, voter_id, vote_value, timestamp, signature).
- Every quorum result (result_id, proposal_id, accept_count, reject_count, abstain_count, outcome, timestamp).
- Every conflict resolution action.
- Every contract violation event.

8.9 Comparison with v1.6 Contract

Aspect	v1.6 Contract (v1.6.3)	v1.7 Contract
Formal specification	Informal documentation	Formally specified with numbered clauses and machine-verifiable invariants
Quorum rules	Partially specified; no minimum_accept_count	Fully specified; quorum_threshold and minimum_accept_count both defined
Conflict resolution	Ad hoc; no defined protocol	Formal conflict resolution protocol with Arbiter support
Vote signing	Optional recommendation	Mandatory; unsigned votes MUST be rejected
Audit log	Best-effort; no schema	Mandatory; standardized schema; 1-second write SLA
Credential rotation	Not defined	Mandatory; 90-day maximum rotation interval
Physics-threat integration	Not present	Consensus suspended during CRITICAL/SUBSTRATE_HALT alerts
Formal invariants	Not present	5 formal invariants (Agreement, Validity, Termination, Non-Duplication, Liveness)

---

§9 — Internal Event Bus (IEB) — v1.7 Additions

9.1 IEB Architecture Summary

The Internal Event Bus (IEB) is the exclusive inter-layer communication substrate for all CK v1.7 components. It is an in-process, priority-channeled, typed event bus. All events are versioned, schema-validated, and carry a trace_id for distributed tracing. The IEB is not a network message broker; it operates within the boundary of a single CK runtime instance. Cross-instance communication for consensus purposes is handled at the Consensus Layer above the IEB.

9.2 Event Schema Standard

All IEB events in v1.7 MUST conform to the following base schema:

```
IEBEvent Base Schema v1.7    event_id          : STRING (UUID v4)
[REQUIRED]   event_type        : STRING (enum value)   [REQUIRED]
schema_version   : STRING ("1.7")        [REQUIRED]   source_component : ENUM
[REQUIRED]     One of: APPLICATION, CONSENSUS, EPOCH_ENGINE, MSL,
PHYSICS_ALERT, IEB_SYSTEM   target_component : ENUM or "BROADCAST"
[REQUIRED]   epoch_id          : STRING                [REQUIRED] (current
epoch at publish time)   trace_id          : STRING (UUID v4)      [REQUIRED]
(for distributed trace correlation)   timestamp_utc    : DATETIME
[REQUIRED] (ISO 8601, millisecond precision)   payload          : OBJECT
[REQUIRED] (event-type-specific; MUST be schema-validated)   priority
: ENUM                  [REQUIRED]     One of: NORMAL, HIGH, CRITICAL, SYSTEM
delivery_class   : ENUM                  [REQUIRED]     One of: AT_MOST_ONCE,
AT_LEAST_ONCE, EXACTLY_ONCE
```

9.3 New Event Types in v1.7

Event Type	Source	Target	Payload Schema (Key Fields)	Delivery Class
MSL_SUBSTRATE_REGISTERED	MSL	BROADCAST	substrate_id, substrate_type, sai_version	AT_LEAST_ONCE
MSL_SUBSTRATE_ACTIVATED	MSL	EPOCH_ENGINE	substrate_id, epoch_id, capability_set	EXACTLY_ONCE
MSL_SUBSTRATE_SUSPENDED	MSL	BROADCAST	substrate_id, reason, alert_id	EXACTLY_ONCE

Event Type	Source	Target	Payload Schema (Key Fields)	Delivery Class
MSL_SUBSTRATE_FAILED	MSL	BROADCAST	substrate_id, error_code, error_detail	EXACTLY_ONCE
PHYSICS_ALERT_PUBLISHED	MSL (PhysicsThreatAlert)	BROADCAST (priority channel)	alert_id, threat_class, severity, substrate_id	EXACTLY_ONCE
EPOCH_ACTIVATED	EPOCH_ENGINE	BROADCAST	epoch_id, substrate_id, start_timestamp	EXACTLY_ONCE
EPOCH_CLOSING	EPOCH_ENGINE	MSL, CONSENSUS	epoch_id, trigger_type, timestamp	EXACTLY_ONCE
EPOCH_FINALIZED	EPOCH_ENGINE	BROADCAST	epoch_id, state_hash, checkpoint_handle, timestamp	EXACTLY_ONCE
EPOCH_ROLLBACK_INITIATED	EPOCH_ENGINE	MSL, CONSENSUS	epoch_id, checkpoint_handle, reason	EXACTLY_ONCE
EPOCH_SUBSTRATE_FAILOVER_INITIATED	MSL	EPOCH_ENGINE	from_substrate_id, to_substrate_id, epoch_id	EXACTLY_ONCE
CONSENSUS_PROPOSAL_SUBMITTED	CONSENSUS	BROADCAST	proposal_id, proposer_id, state_hash	AT_LEAST_ONCE
CONSENSUS_QUORUM_REACHED	CONSENSUS	EPOCH_ENGINE, APPLICATION	proposal_id, outcome, accept_count, reject_count	EXACTLY_ONCE
DESIGN_LAW_VIOLATION	CONSENSUS (DLEE)	BROADCAST	law_id, law_level, violating_component, description	EXACTLY_ONCE
GOVERNANCE_ESCALATION	CONSENSUS	APPLICATION, EXTERNAL	escalation_id, reason, severity, contact_endpoint	EXACTLY_ONCE

9.4 Delivery Semantics

Delivery Class	Guarantee	Use Cases	Implementation Mechanism
AT_MOST_ONCE	Event may be lost; will never be delivered twice.	Advisory telemetry, observability metrics, informational logs.	Fire-and-forget publish. No acknowledgment required.
AT_LEAST_ONCE	Event will be delivered one or more times. Consumer MUST be idempotent.	Governance notifications, substrate lifecycle events, audit triggers.	Publish with acknowledgment; retry on timeout until ACK received. Consumer MUST handle duplicates.
EXACTLY_ONCE	Event will be delivered exactly once, regardless of failures.	Epoch transitions, consensus outcomes, rollback events, PhysicsThreatAlert CRITICAL/HALT.	Two-phase commit with deduplication token. Consumer processes event only if deduplication token is new.

9.5 IEB Backpressure and Flow Control

The IEB MUST implement backpressure to prevent queue overflow and ensure that CRITICAL and SYSTEM priority events are not starved by high volumes of NORMAL priority events. The following mechanisms MUST be implemented:

- **Priority Queue Separation:** Each priority level (NORMAL, HIGH, CRITICAL, SYSTEM) MUST be maintained in a separate internal queue. SYSTEM and CRITICAL queues MUST be processed before HIGH, which MUST be processed before NORMAL.
- **Backpressure Signals:** When a queue depth exceeds the high-watermark threshold (configurable, default: 1000 events per priority level), the IEB MUST publish a BACKPRESSURE_HIGH_WATERMARK event and notify the source component to reduce publish rate.
- **Flow Control for Producers:** Producers MUST implement a configurable publish rate limit. If the IEB signals backpressure, producers MUST reduce their publish rate within 500ms.

- **Dead Letter Queue (DLQ):** Events that cannot be delivered after the maximum retry count MUST be moved to the Dead Letter Queue (§9.6). EXACTLY_ONCE events MUST NOT be silently dropped; DLQ routing is mandatory.

9.6 Dead Letter Queue Policy

The IEB Dead Letter Queue (DLQ) stores events that could not be delivered after exhausting the configured retry policy. DLQ behavior:

- AT_MOST_ONCE events: MAY be dropped instead of DLQ-routed. DLQ routing is not required.
- AT_LEAST_ONCE events: MUST be DLQ-routed after max_retry_count (default: 5) delivery attempts. A DLQ routing event MUST be published to the audit log.
- EXACTLY_ONCE events: MUST be DLQ-routed and MUST trigger a governance escalation event if the event is a CRITICAL or SYSTEM priority type.
- DLQ contents MUST be reviewed by an operator within 1 hour of any EXACTLY_ONCE event entering the DLQ.
- DLQ retention period: 7 days minimum. Events may be replayed from the DLQ by authorized operator action.

9.7 IEB Observability and Tracing

The IEB MUST expose the following observability metrics (see §12.2 for full metric table):

- Events published per second, per priority level, per event_type.
- Queue depth per priority level.
- Delivery latency (p50, p95, p99) per delivery class.
- Dead letter queue depth and DLQ event count per event_type.
- Backpressure signal frequency.
- Retry count distribution per event_type.

All IEB events MUST carry a trace_id field enabling end-to-end distributed tracing of event flows across CK layers. The trace_id MUST be propagated through all derived events generated as a result of a root event.

---

§10 — Design Law (v1.7 Formal Encoding)

10.1 Purpose of Design Law

The CK Design Law is a formally encoded body of architectural principles, constraints, and patterns that govern the design and evolution of all CK components. In CK v1.6, design guidelines existed only as advisory wiki documentation with no enforcement mechanism. CK v1.7 introduces the Design Law Enforcement Engine (DLEE), which formally encodes Design Law and enforces it at runtime through automated checks integrated with the Consensus Layer and IEB.

10.2 Law Hierarchy

Level	Name	Binding	Override Possible?	Enforcement
L1	**Constitutional Law**	Absolute. No exceptions.	No. Requires new CK version.	DLEE MUST reject any design violating Constitutional Law. Violation = conformance failure.
L2	**Statutory Law**	Strong. Deviation requires DLEE-registered waiver.	Yes, via formal waiver process (§10.8).	DLEE MUST log and alert on deviation. Waived deviations are tracked.
L3	**Policy**	Recommended best practice. Team-level approval required to deviate.	Yes, with documented justification.	DLEE SHOULD log deviations. No automated enforcement; human review.

Level	Name	Binding	Override Possible?	Enforcement
L4	**Guideline**	Advisory. Deviations encouraged to be noted but not formally tracked.	Yes, freely.	Not enforced by DLEE.

10.3 v1.7 Constitutional Laws

91. **CL-001 — Layer Isolation:** No CK component at layer N MUST directly invoke or reference internal implementation details of a component at layer N+2 or deeper. All cross-layer communication MUST flow through the IEB and defined interfaces.
92. **CL-002 — Substrate Agnosticism:** The Epoch Engine, Consensus Layer, and Application Layer MUST NOT contain any reference to a specific substrate type, substrate vendor, or substrate hardware model. All substrate interactions MUST be mediated by the MSL.
93. **CL-003 — Audit Completeness:** Every governance decision, state transition, and physics-threat response MUST be recorded in the audit log. No governance action MAY be silently executed.
94. **CL-004 — Determinism:** All CK core algorithms MUST be deterministic. Non-deterministic external inputs (e.g., current wall-clock time, random number generation) MUST be injected as formally typed, logged, and auditable inputs — never read directly from system calls within core algorithms.
95. **CL-005 — Physics Law Supremacy:** Formal physics invariants (§4.8, §6.8, §7.8) MUST take precedence over all application-layer requests. No application-layer directive MAY override a PhysicsThreatAlert-triggered suspension or halt.

10.4 v1.7 Statutory Laws

96. **SL-001 — Schema Versioning:** All IEB events, SAI adapter responses, and audit log records MUST carry an explicit schema_version field. Unversioned data structures are prohibited in inter-component communication.
97. **SL-002 — Idempotency of State-Mutating Operations:** All state-mutating operations exposed by CK components MUST be idempotent when provided the same input and a valid idempotency key. Operations that fail idempotency requirements MUST be wrapped with an idempotency layer before exposure.
98. **SL-003 — Graceful Degradation:** CK components MUST degrade gracefully when a dependency (including a substrate) becomes unavailable. Silent failure is prohibited. Degradation events MUST be published to the IEB and recorded in the audit log.
99. **SL-004 — Credential Lifecycle:** All governance credentials MUST have a defined expiry and renewal process. Expired credentials MUST be rejected. Credential usage MUST be logged per use.
100. **SL-005 — Test Coverage:** All normative requirements (MUST statements) in this specification MUST have a corresponding conformance test in the CK Conformance Test Suite (§12.3). Untested MUST requirements are a statutory law violation.
101. **SL-006 — Backward Compatibility Signaling:** Any component that removes or changes the schema of an existing IEB event MUST increment the schema_version and provide a migration shim for the prior schema version for at minimum one major CK version.

10.5 Prohibited Patterns

Pattern Name	Description	Why Prohibited	Enforcement Mechanism
SUBSTRATE_BYPASS	Any code path that calls substrate	Violates CL-002. Destroys substrate	DLEE static analysis check. Constitutional

Pattern Name	Description	Why Prohibited	Enforcement Mechanism
	hardware directly without going through the MSL/SAI.	agnosticism and prevents physics-threat governance.	violation = immediate conformance failure.
SILENT_GOVERNANCE	A governance decision (epoch transition, consensus vote, alert response) that is executed without being recorded in the audit log.	Violates CL-003. Destroys governance auditability.	DLEE runtime check. Any governance action without audit record triggers DESIGN_LAW_VIOLATION event.
DIRECT_LAYER_SKIP	A component at one layer directly calling into a component two or more layers below, bypassing the IEB.	Violates CL-001. Creates hidden dependencies, breaks isolation, and prevents event tracing.	DLEE dependency graph analysis. Constitutional violation.
NONDETERMINISTIC_CORE	Use of system calls for wall-clock time, random number generation, or OS entropy within core CK algorithms without logging the input value.	Violates CL-004. Prevents deterministic replay and audit verification.	DLEE static analysis. Statutory violation with mandatory waiver if justified.
UNVERSIONED_SCHEMA	IEB events or SAI adapter responses that lack a schema_version field.	Violates SL-001. Prevents schema migration and compatibility management.	IEB schema validator rejects unversioned events at publish time.
CREDENTIAL_REUSE_AFTER_EXPIRY	Use of an expired governance credential for any purpose.	Violates SL-004. Creates security vulnerability in governance chain.	Credential validator in Consensus Layer. MUST reject expired credentials.

10.6 Approved Patterns

Pattern Name	Description	When to Use
EVENT_SOURCED_STATE	Derive component state exclusively from the ordered sequence of IEB events, never from direct state mutation.	Wherever deterministic state replay and audit are required.
IDEMPOTENT_HANDLER	Design all IEB event consumers to be safely callable multiple times with	All AT_LEAST_ONCE and EXACTLY_ONCE event consumers.

Pattern Name	Description	When to Use
	the same event, using an event_id-based deduplication check.	
CIRCUIT_BREAKER_FOR_SUBSTRATE	Wrap all SAI execute() calls with a circuit breaker pattern to prevent cascade failure when a substrate degrades.	All MSL-to-substrate SAI adapter invocations.
CHECKPOINT_BEFORE_RISKY_OP	Proactively checkpoint the current epoch state before any operation that carries elevated physics-threat risk (e.g., long quantum circuits near T2 boundary).	Any Epoch Engine or MSL operation that may trigger a PhysicsThreatAlert.
TYPED_TELEMETRY_SCHEMA	Define a typed, versioned schema for all physics telemetry payloads, specific to each substrate type.	All substrate SAI adapter physics telemetry implementations.

10.7 Design Law Enforcement Mechanisms

The Design Law Enforcement Engine (DLEE) implements enforcement through the following mechanisms:

- **Static Analysis Integration:** DLEE provides a static analysis plugin (compatible with major build toolchains) that checks Constitutional and Statutory Law compliance at build time. Build MUST fail on Constitutional Law violations.
- **Runtime Monitoring:** DLEE hooks into the IEB to monitor events for design law violations at runtime. Detected violations MUST publish a DESIGN_LAW_VIOLATION event to the IEB.
- **Audit Log Integration:** All DESIGN_LAW_VIOLATION events MUST be written to the audit log with full context (violating component, law_id, description, timestamp).
- **Governance Integration:** DESIGN_LAW_VIOLATION events of Constitutional severity MUST trigger a governance escalation event.

10.8 Variance and Waiver Process

Statutory Law violations MAY be authorized via a formal waiver process:

102. The requesting team submits a Variance Request to the CK Architecture Council, including: law_id, description of the deviation, technical justification, mitigations in place, and duration of the waiver request.

103. The Architecture Council reviews the Variance Request within 10 business days.

104. If approved, a Waiver Record is created in the DLEE Waiver Registry with a unique waiver_id, expiry date, and associated law_id.

105. The waiver_id MUST be referenced in the code or configuration where the deviation occurs.

106. Waivers expire on their stated expiry date. Expired waivers MUST be renewed or the deviation corrected. Expired waivers return the code to a Statutory violation state.

107. Constitutional Laws MUST NOT be waived under any circumstances.

---

§11 — Migration Guide: v1.6 → v1.7

11.1 Migration Overview and Timeline

Migration from CK v1.6.x to CK v1.7.0 is a major version migration. Due to the breaking changes introduced in v1.7 (see §3), migration MUST be carefully planned and executed. The expected migration timeline for a typical production deployment is 4–8 weeks, depending on the number of active substrates, the complexity of the v1.6 epoch engine integration, and the degree of IEB schema customization in the existing deployment.

Migration Warning

Migration from CK v1.6 to CK v1.7 is NOT backward compatible. A CK v1.7 runtime MUST NOT attempt to connect or synchronize with a CK v1.6 runtime instance within the same consensus group. All instances in a consensus group MUST be migrated together in a coordinated cutover.

11.2 Pre-Migration Checklist

#	Checklist Item	Owner	Status
1	Identify all CK v1.6 runtime instances in the deployment (node inventory).	Infrastructure Lead	[ ] Complete
2	Document current substrate configurations (type, hardware model, SAI adapter version).	Architecture Lead	[ ] Complete
3	Review the §3 version delta table and identify all breaking changes that apply to the specific deployment.	Architecture Lead	[ ] Complete
4	Ensure all current epochs are in FINALIZED state before migration. No epoch MUST be in ACTIVE or CLOSING state at migration initiation.	Operations Lead	[ ] Complete
5	Back up all audit log records from the v1.6 deployment.	Compliance Officer	[ ] Complete
6	Obtain and install the CK v1.7 migration toolchain (§11.4).	Engineering Lead	[ ] Complete
7	Run the pre-migration checker tool and resolve all reported issues.	Engineering Lead	[ ] Complete
8	Prepare and test new SAI adapters for	Engineering Lead	[ ] Complete

#	Checklist Item	Owner	Status
	all substrates to be used in v1.7 deployment.		
9	Prepare new Consensus Contract v1.7 configuration (quorum settings, governance credentials).	Architecture Lead	[ ] Complete
10	Schedule maintenance window. Notify all consumers of the Application Layer interface.	Operations Lead	[ ] Complete
11	Verify rollback procedure (§11.6) is documented and the team is trained.	Operations Lead	[ ] Complete
12	Confirm all custom IEB event schemas have been reviewed against v1.7 IEB schema standard.	Engineering Lead	[ ] Complete

11.3 Breaking Changes and Required Adaptations

Breaking Change	Required Adaptation
Epoch Engine substrate coupling removed	All substrate-specific logic MUST be moved to SAI adapters. Epoch Engine MUST be updated to communicate via MSL IEB interface.
Consensus Contract v1.6.3 → v1.7	Consensus configuration must be replaced. All validators must be updated to v1.7 contract signing and quorum behavior. v1.6 vote formats will be rejected.
IEB event schema changes	All custom IEB event publishers and consumers MUST add schema_version field. Consumers MUST implement idempotency via event_id deduplication.
Audit log now mandatory	If v1.6 deployment ran without audit logging, a compliant audit log backend MUST be provisioned and configured before v1.7 runtime starts.
Design Law enforcement now active	DLEE static analysis MUST be run against all

Breaking Change	Required Adaptation
	existing codebase. Constitutional Law violations MUST be resolved before deployment.
PostLattice_v1 deprecated	MUST migrate to PostLattice_v2 using the defined upgrade path (§5.2.6). PostLattice_v1 adapters will not load in CK v1.7.

11.4 Migration Toolchain

The CK v1.7 migration toolchain provides the following tools:

- `ck-migrate check` — Pre-migration checker. Scans the v1.6 deployment configuration and codebase for breaking change issues. Reports findings with severity classifications.
- `ck-migrate substrate upgrade --from [source] --to [target]` — Substrate adapter migration tool. Generates a PostLattice_v2 configuration from an existing PostLattice_v1 configuration.
- `ck-migrate schema migrate --schema-dir [dir]` — IEB schema migrator. Adds schema_version fields and deduplication key stubs to v1.6 event schemas.
- `ck-migrate audit export --format v1.7` — Exports v1.6 audit log records in v1.7-compatible format for audit continuity.
- `ck-migrate validate --suite conformance` — Runs the post-migration validation test suite (§11.7) against the v1.7 deployment.

11.5 Step-by-Step Migration Procedure

108. Complete all pre-migration checklist items (§11.2).

109. Enter scheduled maintenance window. Halt all application-layer workloads.

110. Finalize any remaining ACTIVE epochs on the v1.6 runtime. Verify all epochs are in FINALIZED state.

111. Flush and back up the v1.6 audit log.

112. Shut down all CK v1.6 runtime instances in the consensus group simultaneously.
113. Install CK v1.7 binaries on all nodes.
114. Deploy new SAI adapters for all substrates (QIGEM_v1 and/or PostLattice_v2).
115. Configure the MSL with substrate registrations and capability matrix settings.
116. Configure the Consensus Contract v1.7 (quorum settings, governance credentials, arbiter if applicable).
117. Configure IEB with updated event schemas and delivery class settings.
118. Configure PhysicsThreatAlert thresholds for all substrates.
119. Configure DLEE enforcement settings and load the waiver registry.
120. Start CK v1.7 runtime instances. Observe BOOTSTRAP and SUBSTRATE_READY lifecycle phases.
121. Verify all substrates reach STANDBY state without CAPABILITY_MISMATCH events.
122. Activate the first CK v1.7 epoch. Verify PENDING → ACTIVE transition.
123. Run the post-migration validation test suite (§11.7).
124. If all validation tests pass, resume application-layer workloads. Exit maintenance window.
125. Monitor observability dashboards for 24 hours post-migration.

11.6 Rollback Procedure

If migration fails at any step, execute the following rollback procedure:

126. Halt all CK v1.7 instances immediately.
127. Do NOT attempt to finalize any epoch on the v1.7 runtime if migration validation failed.
128. Restore the v1.6 audit log backup.

129. Reinstall CK v1.6 binaries on all nodes.

130. Restore the v1.6 substrate and consensus configurations.

131. Start CK v1.6 runtime instances and verify OPERATIONAL state.

132. Resume application-layer workloads on the v1.6 runtime.

133. Document the failure, including the step at which migration failed and the observed error condition, and submit a failure report to the CK Architecture Council.

11.7 Post-Migration Validation Tests

The following validation tests MUST pass before the maintenance window is closed:

134. MSL substrate registration and STANDBY state for all configured substrates.

135. PhysicsThreatAlert pipeline produces a synthetic ADVISORY alert when injected with test telemetry below threshold.

136. Epoch lifecycle: PENDING → ACTIVE → CLOSING → FINALIZED transitions complete without error.

137. Checkpoint creation and verified checkpoint handle returned for a test epoch.

138. Consensus Contract v1.7 quorum: three-validator proposal and acceptance cycle completes.

139. IEB delivers an EXACTLY_ONCE event with correct deduplication behavior (duplicate suppressed on retry).

140. Audit log contains records for all epoch transitions, consensus events, and MSL lifecycle events generated during validation.

141. DLEE reports no Constitutional Law violations in the deployed configuration.

11.8 Known Migration Issues and Workarounds

Issue	Symptom	Workaround
SAI adapter load failure after upgrade	MSL_SUBSTRATE_FAILED event on BOOTSTRAP; adapter exits immediately.	Check adapter binary compatibility with CK v1.7 SAI host ABI. Rebuild adapter against v1.7 SAI headers.
v1.6 audit log format incompatible	Audit log reader in v1.7 rejects records from v1.6 export.	Use `ck-migrate audit export --format v1.7` to convert records before import.
IEB consumers receiving duplicate EXACTLY_ONCE events	Consumers processing the same event twice after restart.	Ensure consumers implement event_id-based deduplication. Check DLQ for unacknowledged events.
Consensus quorum not reachable during cutover	CONSENSUS_QUORUM_LOST event; epoch stuck in PENDING.	Verify all v1.7 validator instances are running and governance credentials are valid. Check network connectivity between nodes.
PhysicsThreatAlert false positives on QIGEM_v1	COHERENCE_COLLAPSE ADVISORY immediately after activation.	Hardware warmup period required. Increase t1_min_us threshold temporarily by 10%; reduce after 10 minutes of stable operation. Submit substrate recalibration request.
DLEE reports false Constitutional violations	Build fails on SUBSTRATE_BYPASS detection in legacy adapter shim code.	Refactor shim code to route through MSL SAI interface. Contact Architecture Council if shim is required by hardware vendor.

---

§12 — Observability, Testing & Compliance

12.1 Observability Requirements

A conformant CK v1.7 runtime MUST expose a standard observability interface providing access to metrics, structured logs, and distributed traces. The

observability interface MUST be accessible to external monitoring systems without requiring access to internal runtime state. All observability data MUST be produced with at most 5-second latency from the events they describe.

12.2 Required Metrics

Metric Name	Type	Description	Alert Threshold
ck_epoch_transitions_total	Counter	Total epoch lifecycle transitions, labeled by transition type.	—
ck_epoch_transition_duration_ms	Histogram	Duration of epoch lifecycle transitions.	p99 > 10,000ms = WARNING
ck_epoch_rollback_total	Counter	Total epoch rollbacks initiated.	Any increment = WARNING
ck_physics_alert_total	Counter	Total PhysicsThreatAlert events, labeled by threat_class and severity.	Any CRITICAL = alert
ck_substrate_health_status	Gauge	Current health status of each substrate (0=FAILED, 1=DEGRADED, 2=OK).	< 2 for any ACTIVE substrate
ck_ieb_queue_depth	Gauge	Current IEB queue depth per priority level.	> 800 events = WARNING
ck_ieb_delivery_latency_ms	Histogram	Event delivery latency per delivery class.	EXACTLY_ONCE p99 > 500ms = WARNING
ck_ieb_dlq_depth	Gauge	Dead Letter Queue depth.	> 0 = WARNING
ck_consensus_quorum_ratio	Gauge	Current ratio of ACCEPT votes to total votes in most recent consensus round.	< 0.67 = WARNING
ck_audit_log_write_latency_ms	Histogram	Audit log write latency.	p99 > 1,000ms = WARNING
ck_msl_substrate_lifecycle_events_total	Counter	Total MSL substrate lifecycle events, labeled by event type.	MSL_SUBSTRATE_FAILED increment = CRITICAL
ck_design_law_violat	Counter	Total DLEE-detected	Any Constitutional =

Metric Name	Type	Description	Alert Threshold
ions_total		Design Law violations, labeled by law_level.	CRITICAL

12.3 Conformance Test Suite Overview

The CK Conformance Test Suite (CTS) is a mandatory test suite that all CK v1.7 runtime implementations MUST pass to claim conformance. The CTS is organized into the following test categories:

- **CTS-MSL:** MSL lifecycle tests, SAI interface compliance tests, capability matrix enforcement tests.
- **CTS-EPOCH:** Epoch state machine tests (all transitions and guards), checkpoint and rollback tests, performance target validation.
- **CTS-PTA:** PhysicsThreatAlert pipeline tests, alert schema validation, severity classification tests, escalation policy tests.
- **CTS-CONSENSUS:** Consensus Contract v1.7 term enforcement tests, quorum rule tests, conflict resolution tests, audit log completeness tests.
- **CTS-IEB:** Delivery semantics tests (all three delivery classes), backpressure tests, DLQ routing tests, schema validation tests.
- **CTS-DLAW:** Design Law Enforcement Engine tests, Constitutional and Statutory law detection tests, waiver registry tests.
- **CTS-AUDIT:** Audit log completeness, tamper-evidence, and retention tests.
- **CTS-SUBSTRATE:** QIGEM_v1 and PostLattice_v2 substrate-specific compliance tests.

CTS results MUST be submitted to the CK Architecture Council as part of the conformance attestation process.

12.4 Compliance Attestation Process

142. The implementing team runs the full CTS against the deployed CK v1.7 runtime.

143. CTS results (test name, pass/fail, execution timestamp, runtime version) are packaged into a Conformance Report.

144. The Conformance Report is submitted to the CK Architecture Council via the designated submission channel.

145. The Architecture Council reviews the report within 15 business days.

146. If all mandatory CTS tests pass, the implementation is issued a Conformance Certificate valid for 12 months.

147. Conformance Certificates are subject to quarterly spot-check audits. Failures in spot-check audits result in certificate suspension pending remediation.

12.5 Audit Trail Requirements

The CK v1.7 audit trail MUST satisfy the following requirements:

- **Completeness:** All events listed in the audit log requirements sections of §8.8, §7.8, §10.7, and §9.6 MUST be recorded.
- **Tamper-Evidence:** Audit log records MUST be hash-chained (each record's hash includes the hash of the preceding record) to detect tampering. The chain root hash MUST be externally witnessed at minimum every 24 hours.
- **Write Latency:** Audit log writes MUST complete within 1 second of the event they record. Writes that fail MUST be retried; a write failure that persists for more than 5 seconds MUST trigger a GOVERNANCE_ESCALATION event.
- **Retention Period:** Audit log records MUST be retained for a minimum of 7 years. Records for SUBSTRATE_HALT events and associated epochs MUST be retained for a minimum of 10 years.
- **Queryability:** The audit log MUST support indexed queries by epoch_id, substrate_id, event_type, timestamp range, and alert_id.
- **Access Control:** Audit log records MUST be write-once (append-only). Deletion of audit log records is prohibited except by authorized archive transfer to compliant long-term storage. Read access MUST be role-restricted.

§13 — Roadmap: CK v1.8 Preview

13.1 Themes for v1.8

CK v1.8 is currently in the early planning phase. The primary architectural themes for v1.8 are:

- **Dynamic Substrate Management:** Enable hot-swap of substrates without epoch finalization, enabling higher availability for long-running cognitive workloads.
- **Parallelism and Concurrency:** Enable multiple epochs to execute concurrently on distinct substrates, dramatically increasing throughput for parallelizable workloads.
- **Formal Verification Integration:** Introduce machine-checkable proofs for critical CK invariants, replacing runtime assertion checks with pre-verified correctness guarantees.
- **Ecosystem Openness:** Define a Substrate Marketplace Protocol enabling third-party substrate vendors to publish and certify SAI-compliant substrate adapters.
- **Adaptive Physical Calibration:** Introduce a feedback loop between PhysicsThreatAlert telemetry and substrate operating parameters, enabling autonomous physics calibration without human intervention for ADVISORY and WARNING conditions.

13.2 Planned Features

Feature	Description	Target Milestone	Dependency
Dynamic Substrate Hot-Swap	Enable replacement of an ACTIVE substrate with a new substrate without requiring epoch	CK v1.8.0	SAI v1.8 HOT_SWAP capability; MSL hot-swap coordinator; enhanced PhysicsThreatAlert

Feature	Description	Target Milestone	Dependency
	finalization. State is migrated via a live checkpoint-and-restore cycle.		monitoring during swap.
Multi-Epoch Parallelism	Allow multiple epochs to be simultaneously in ACTIVE state on distinct substrate assignments. Requires new parallel consensus protocol.	CK v1.8.0	Parallel Consensus Contract v1.8; Epoch Engine parallel scheduler; IEB ordering guarantees for parallel epoch events.
Formal Verification Integration	Integrate with a formal verification toolchain (TLA+, Coq, or Lean) to produce machine-checked proofs of key invariants (Agreement, Validity, Termination).	CK v1.8.1	Formal model of Consensus Contract and Epoch State Machine; verification toolchain CI integration.
Substrate Marketplace Protocol	Define an open protocol for third-party substrate vendors to publish SAI-compliant adapters, including a certification process and a signed adapter registry.	CK v1.8.1	SAI v1.8 compatibility; adapter signing infrastructure; marketplace registry service.
Adaptive Physics Calibration	Introduce an autonomous calibration feedback loop: PhysicsThreatAlert ADVISORY and WARNING signals are fed back to substrate operating parameter adjustment, reducing human intervention requirements.	CK v1.8.2	Substrate parameter write-back API in SAI v1.8; DLEE approval for autonomous calibration actions; PhysicsThreatAlert calibration action audit trail.

13.3 Deprecation Schedule

Component	Deprecated In	Removed In	Replacement
PostLattice_v1 Substrate	CK v1.7.0	CK v1.8.0	PostLattice_v2
IEB Schema v1.6 (unversioned)	CK v1.7.0	CK v1.8.0	IEB Schema v1.7 (versioned)
Consensus Contract v1.6.3	CK v1.7.0	CK v1.8.0	Consensus Contract v1.7
CK v1.6 Epoch Engine API	CK v1.7.0	CK v1.8.0	CK v1.7 MSL-mediated Epoch Engine interface
SAI v1.6 (pre-MSL adapter interface)	CK v1.7.0	CK v1.8.0	SAI v1.7

13.4 Community RFC Process

CK v1.8 features are developed through a Request for Comments (RFC) process managed by the CK Architecture Council. The RFC process enables implementers, substrate vendors, and governance stakeholders to contribute to the evolution of the CK specification.

- RFC submissions are accepted on a rolling basis via the designated RFC repository.
- Each RFC MUST specify: feature title, motivation, proposed specification changes (with section references), migration impact, and required new invariants.
- RFCs are reviewed by the Architecture Council within 30 days of submission.
- Accepted RFCs are assigned to a target milestone and their specification language is incorporated into the relevant specification draft.
- All RFC authors MUST agree to assign specification copyright to the CK Architecture Council upon RFC acceptance.

Appendix A — Schema Reference

A.1 SubstrateDescriptor

Field	Type	Required	Description	Constraints
substrate_id	STRING	Yes	Unique substrate instance identifier.	Format: [type]_[version]_[uuid4]. Immutable after registration.
substrate_type	ENUM	Yes	Substrate class.	One of: QUANTUM, LATTICE_PQ, CLASSICAL, HYBRID.
sai_version	STRING	Yes	SAI interface version implemented.	MUST be "1.7" or later for CK v1.7.
capabilities	CAPABILITY_SET	Yes	Declared capabilities of this substrate.	MUST include all REQUIRED capabilities from §4.5.
hardware_profile	OBJECT	Yes	Hardware metadata: vendor, hw_model, firmware_version, topology.	Substrate-type-specific schema.
lifecycle_state	ENUM	Yes (runtime)	Current MSL lifecycle state.	One of: REGISTERED, VALIDATING, STANDBY, ACTIVE, SUSPENDED, DEACTIVATED, FAILED.
registration_timestamp	DATETIME (UTC)	Yes	Timestamp of MSL registration.	ISO 8601, millisecond precision. Immutable.
fallback_substrate_id	STRING	No	substrate_id of the designated fallback substrate.	If set, MUST reference a registered substrate_id.
role	ENUM	Yes	Assigned role in multi-substrate configuration.	One of: PRIMARY, FALLBACK, ANCILLARY.

A.2 EpochRecord

Field	Type	Required	Description	Constraints
epoch_id	STRING	Yes	Unique epoch identifier.	Format: {runtime_id}:{epoch_seq}:{utc_timestamp}. Globally unique.
state	ENUM	Yes	Current epoch lifecycle state.	One of: PENDING, ACTIVE, CLOSING, FINALIZED, ARCHIVED, SUSPENDED.
substrate_id	STRING	Yes	Primary substrate assigned to this epoch.	MUST reference an ACTIVE substrate at epoch ACTIVE transition.
start_timestamp	DATETIME (UTC)	Yes	Epoch ACTIVE state entry timestamp.	ISO 8601, millisecond precision.
end_timestamp	DATETIME (UTC)	No	Epoch FINALIZED state entry timestamp.	Set at FINALIZED transition. NULL for non-finalized epochs.
state_hash	STRING (SHA-256 hex)	Yes (FINALIZED)	SHA-256 hash of the epoch's final state vector.	64-character hex string. Set at FINALIZED transition.
checkpoint_handle	OBJECT	Yes (FINALIZED)	CHECKPOINT_HANDLE returned by MSL for this epoch.	Contains: epoch_id, sequence_number, state_hash, substrate_id, creation_timestamp_utc.
workload_count	INTEGER	Yes	Total workloads admitted in this epoch.	≥ 0.
rollback_eligible	BOOLEAN	Yes	Whether this epoch is eligible for rollback.	True if checkpoint_handle is valid and not expired.
consensus_proposal_id	STRING (UUID v4)	Yes (FINALIZED)	Consensus proposal ID for	Set at FINALIZED

Field	Type	Required	Description	Constraints
			the finalization vote.	transition.

A.3 PhysicsThreatAlertRecord

See §7.4 for the full PhysicsThreatAlert schema. The following additional fields are present in the stored audit record (not present in the IEB event):

Field	Type	Required	Description	Constraints
audit_record_id	STRING (UUID v4)	Yes	Unique audit record identifier (distinct from alert_id).	Generated by audit log writer. Immutable.
audit_write_timestamp	DATETIME (UTC)	Yes	Timestamp of audit log write (not alert generation).	MUST be within 1 second of alert generation timestamp.
response_outcome	STRING	Yes	Final outcome of the alert response (human or automated).	Set after response completes. One of: RESOLVED, ESCALATED, SUBSTRATE_REPLACED, ROLLBACK_EXECUTED, PENDING.
resolution_timestamp	DATETIME (UTC)	No	Timestamp when the alert was resolved.	NULL until RESOLVED or terminal outcome recorded.
human_responder_id	STRING	No	Identifier of the human responder (governance officer).	Required if human_escalation_required was true.

A.4 ConsensusContractRecord

Field	Type	Required	Description	Constraints
proposal_id	STRING (UUID	Yes	Unique	Immutable after

Field	Type	Required	Description	Constraints
	v4)		consensus proposal identifier.	creation.
proposer_id	STRING	Yes	Runtime instance identifier of the proposer.	MUST hold a valid PROPOSER governance credential.
epoch_id	STRING	Yes	Epoch ID this proposal is associated with.	MUST reference a valid epoch in CLOSING state.
proposed_state_hash	STRING (SHA-256 hex)	Yes	Proposed epoch state hash.	MUST be a valid state transition from prior_state_hash.
prior_state_hash	STRING (SHA-256 hex)	Yes	State hash of the preceding finalized epoch.	MUST match the state_hash of the immediately prior EpochRecord.
proposer_signature	STRING (Base64)	Yes	Cryptographic signature of the proposal by the proposer.	MUST be verifiable against the proposer's current governance credential.
outcome	ENUM	Yes	Final proposal outcome.	One of: PENDING, ACCEPTED, REJECTED, LAPSED, CONFLICTED.
accept_count	INTEGER	Yes	Number of ACCEPT votes at outcome determination.	≥ 0.
reject_count	INTEGER	Yes	Number of REJECT votes at outcome determination.	≥ 0.
creation_timestamp	DATETIME (UTC)	Yes	Proposal creation timestamp.	ISO 8601, millisecond precision.
outcome_timestamp	DATETIME (UTC)	No	Timestamp of outcome determination.	NULL for PENDING proposals.

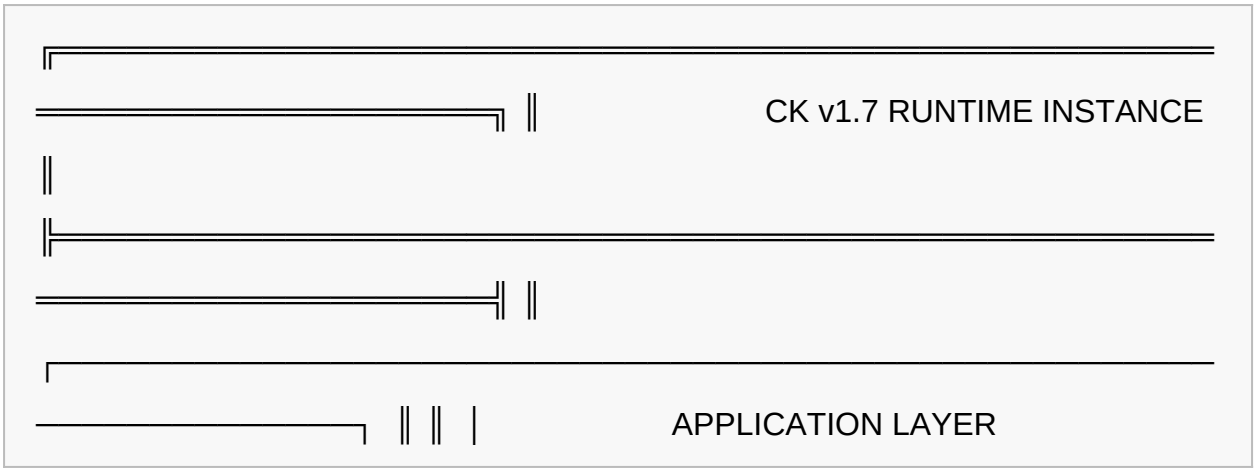
A.5 IEBEvent

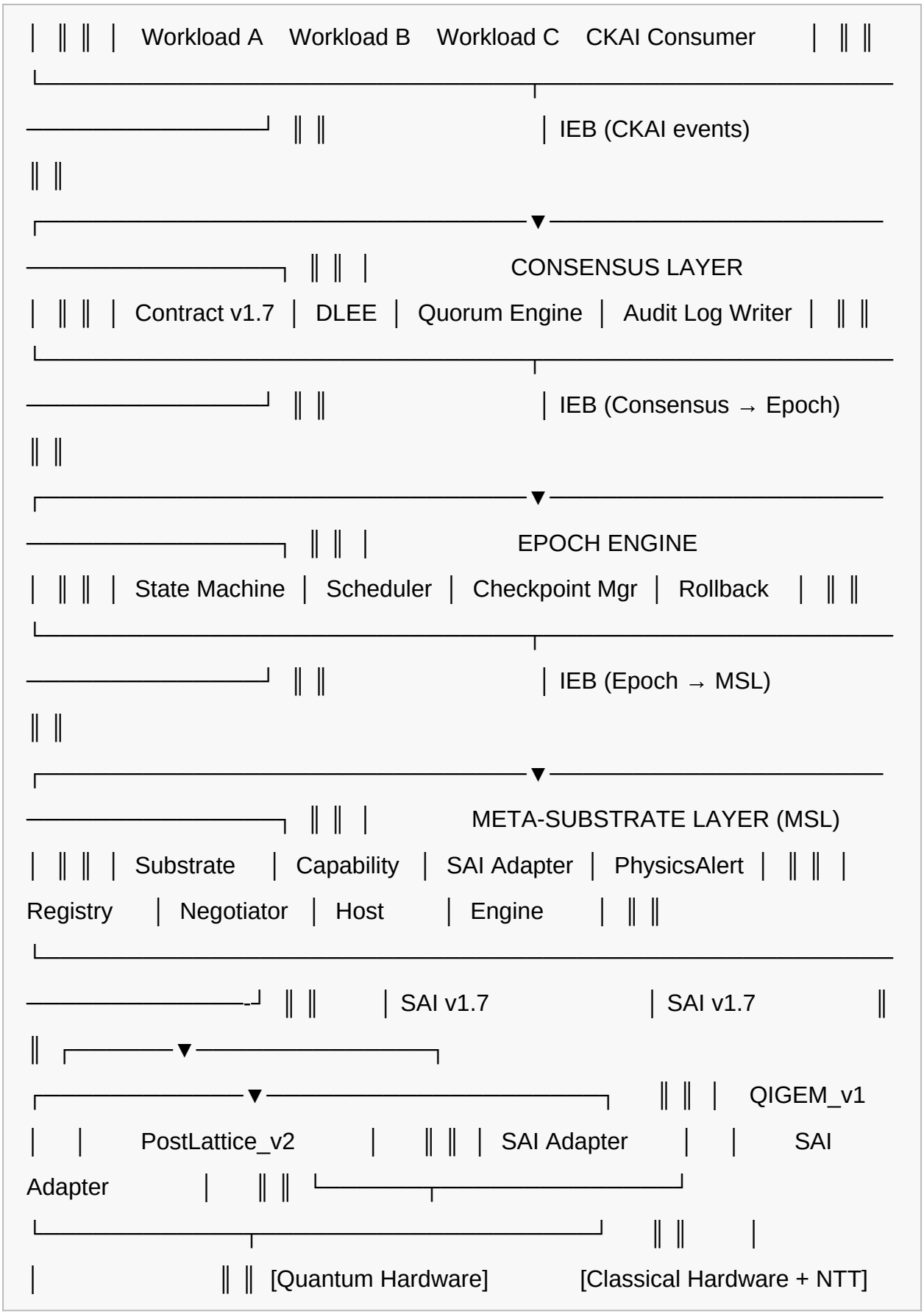
See §9.2 for the full IEBEvent base schema. The stored audit record extends the base schema with:

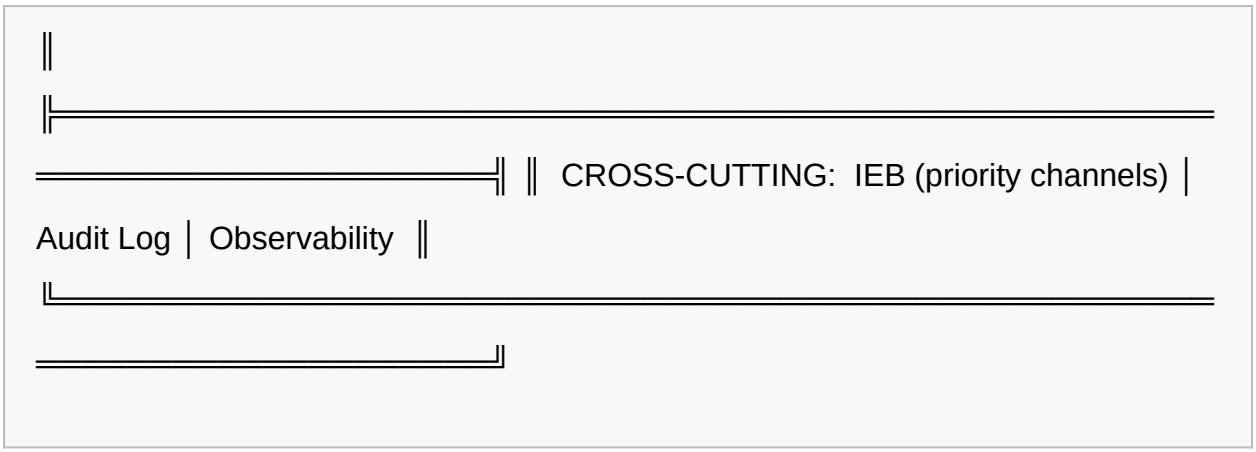
Field	Type	Required	Description	Constraints
delivery_attempts	INTEGER	Yes	Number of delivery attempts made.	≥ 1 for any event that was published.
delivery_status	ENUM	Yes	Final delivery status.	One of: DELIVERED, DLQ_ROUTED, DROPPED (AT_MOST_ONCE only).
consumer_id	STRING	Yes	Identifier of the consuming component.	MUST match a registered IEB consumer.
deduplication_token	STRING (UUID v4)	Yes (EXACTLY_ONCE)	Deduplication token for EXACTLY_ONCE events.	Required for EXACTLY_ONCE delivery class.

Appendix B — Architecture Diagrams

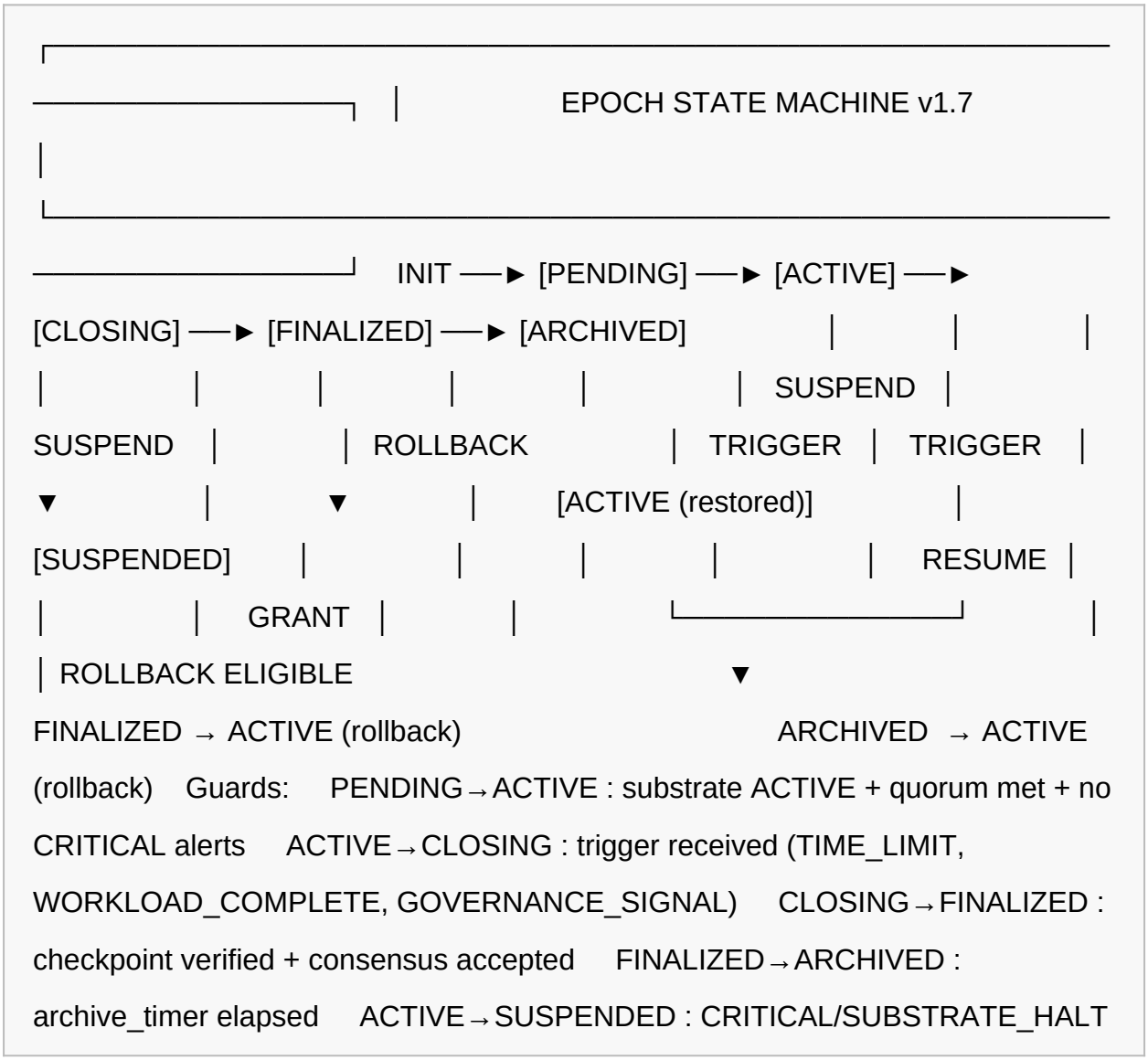
B.1 Full Layer Stack with MSL




Workload A
Workload B
Workload C
CKAI Consumer
IEB (CKAI events)
CONSENSUS LAYER
Contract v1.7
DLEE
Quorum Engine
Audit Log Writer
IEB (Consensus → Epoch)
EPOCH ENGINE
State Machine
Scheduler
Checkpoint Mgr
Rollback
IEB (Epoch → MSL)
META-SUBSTRATE LAYER (MSL)
Substrate
Capability
SAI Adapter
PhysicsAlert
Registry
Negotiator
Host
Engine
SAI v1.7
SAI v1.7
QIGEM_v1
PostLattice_v2
SAI Adapter
SAI Adapter
[Quantum Hardware]
[Classical Hardware + NTT]

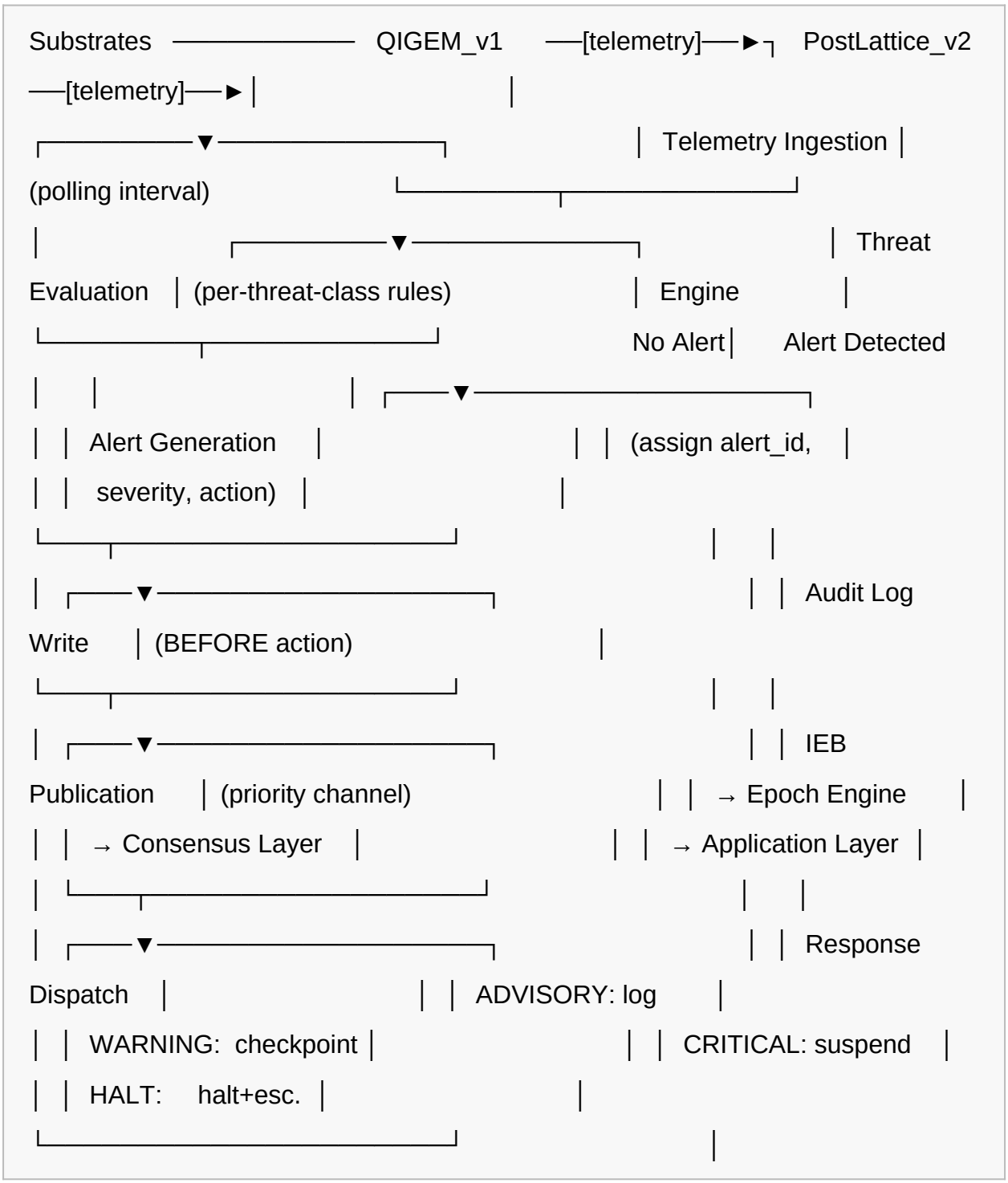


B.2 Epoch State Machine



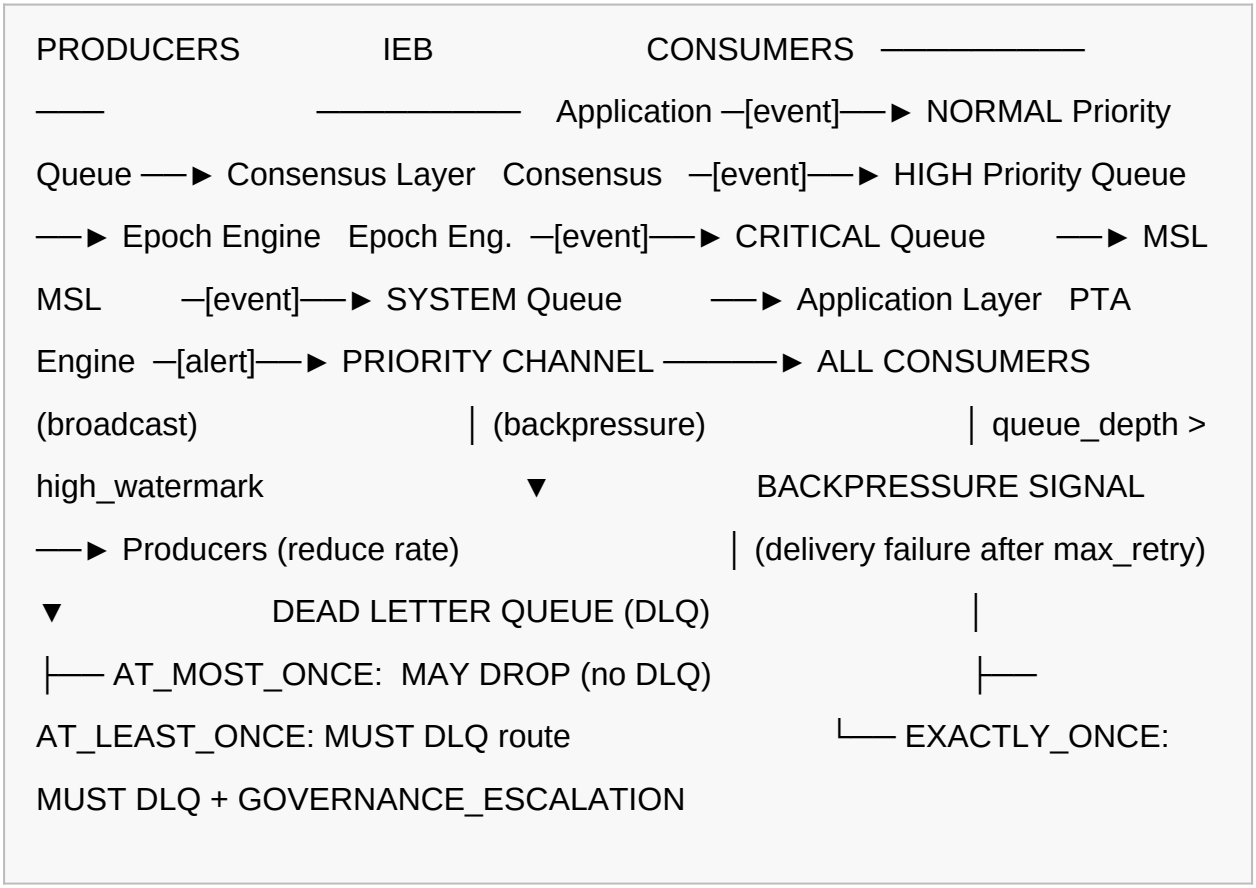
alert received SUSPENDED→ACTIVE : governance clearance received

B.3 PhysicsThreatAlert Pipeline



```
[continue monitoring]
```

B.4 IEB Message Flow



Appendix C — Formal Invariants Summary

The following is the master numbered list of all formally stated invariants defined across this specification, tagged by their originating section.

#	Invariant ID	Section	Statement (Summary)
1	MSL-INV-001	§4.8	Only ACTIVE

#	Invariant ID	Section	Statement (Summary)
			substrates MAY accept execute() calls.
2	MSL-INV-002	§4.8	Every substrate MUST complete VALIDATING before entering STANDBY or ACTIVE.
3	MSL-INV-003	§4.8	Exactly one SAI adapter instance per registered substrate at all times.
4	MSL-INV-004	§4.8	Physics telemetry MUST be collected from every ACTIVE substrate at least once per polling interval.
5	MSL-INV-005	§4.8	Substrate registry entries MUST NOT be removed after registration.
6	MSL-INV-006	§4.8	All MSL lifecycle state transitions MUST be published to IEB and audit log before completion.
7	MSL-INV-007	§4.8	Substrate-specific internals MUST NOT be exposed above the MSL except through defined MSL event schema.
8	MSL-INV-008	§4.8	MSL component failure MUST trigger SUBSTRATE_HALT protocol.
9	EE-INV-001	§6.8	At most one epoch SHALL be in ACTIVE or CLOSING state at any given time per runtime instance.
10	EE-INV-002	§6.8	No epoch SHALL transition PENDING→ACTIVE without at least one ACTIVE substrate.
11	EE-INV-003	§6.8	No epoch SHALL

#	Invariant ID	Section	Statement (Summary)
			transition to FINALIZED without a verified, non-null checkpoint handle.
12	EE-INV-004	§6.8	Epoch identifiers MUST be globally unique and monotonically increasing.
13	EE-INV-005	§6.8	ARCHIVED epochs MAY be rolled back; rolled-back epochs MUST be re-archived after resolution.
14	EE-INV-006	§6.8	Epoch transition events MUST be published to IEB before local state machine records the transition.
15	EE-INV-007	§6.8	The Epoch Engine MUST NOT reference any specific substrate type.
16	EE-INV-008	§6.8	No execute() calls SHALL be dispatched by the Epoch Engine during SUSPENDED state.
17	PTA-INV-001	§7.8	Every telemetry collection cycle MUST include a threat evaluation pass.
18	PTA-INV-002	§7.8	All alerts MUST be written to audit log before any automated response action.
19	PTA-INV-003	§7.8	SUBSTRATE_HALT alerts MUST halt all substrate execute() calls within 100ms.
20	PTA-INV-004	§7.8	Alert records MUST be immutable after generation.
21	PTA-INV-005	§7.8	PhysicsThreatAlert Engine

#	Invariant ID	Section	Statement (Summary)
			unresponsiveness MUST be treated as SUBSTRATE_HALT.
22	PTA-INV-006	§7.8	human_escalation_required MUST be true for every SUBSTRATE_HALT alert.
23	CC-INV-001	§8.4	Agreement: All accepting validators converge to identical state hashes.
24	CC-INV-002	§8.4	Validity: Accepted state hashes MUST be valid transitions from the prior accepted state.
25	CC-INV-003	§8.4	Termination: Every proposal MUST eventually reach ACCEPTED, REJECTED, or LAPSED.
26	CC-INV-004	§8.4	Non-Duplication: No epoch state hash SHALL be the accepted hash of more than one epoch.
27	CC-INV-005	§8.4	Liveness: Without exceeding Byzantine fault threshold, proposals MUST resolve within 2× timeout.

Appendix D — Glossary

Term	Definition	First Used In Section
ARBITER	A designated CK runtime instance or external governance authority that	§8.2

Term	Definition	First Used In Section
	resolves tie votes in the consensus mechanism.	
Audit Log	The tamper-evident, append-only record of all governance decisions, state transitions, and physics-threat events in a CK runtime instance.	§0.1
Checkpoint	An atomic, verifiable snapshot of epoch state created by the MSL via the SAI checkpoint() call, identified by a CHECKPOINT_HANDLE.	§6.7
CHECKPOINT_HANDLE	A structured identifier for a substrate checkpoint, containing: epoch_id, sequence_number, state_hash, substrate_id, creation_timestamp_utc.	§6.7
CK	Cognitive Kernel. The platform defined by this specification.	§0.1
CKAI	CK Application Interface. The interface between the Application Layer and the Consensus Layer.	§2.4
Constitutional Law	The highest level of Design Law, absolutely binding with no possibility of waiver.	§10.2
Consensus Contract	The formal agreement governing distributed state agreement among CK runtime instances.	§0.1
Dead Letter Queue (DLQ)	The IEB storage for events that could not be delivered after exhausting the retry policy.	§9.6
Design Law	The formally encoded body of architectural principles governing CK component design and evolution.	§0.1
DLEE	Design Law Enforcement Engine. The runtime and static analysis system enforcing Design Law.	§10.7
Epoch	A formally bounded, checkpointed, auditable	§6.1

Term	Definition	First Used In Section
	segment of CK runtime operational history.	
Epoch Engine	The CK component managing the lifecycle of epochs: scheduling, transitions, checkpointing, and rollback.	§0.1
Gate Fidelity	The accuracy of a quantum gate operation, measured as the probability of the gate producing the correct output state.	§5.1.4
IEB	Internal Event Bus. The exclusive inter-layer communication channel in a CK runtime instance.	§2.3
Invariant	A formally stated property that MUST hold true at all times in a conformant CK implementation.	§0.1
MSL	Meta-Substrate Layer. The CK architectural layer providing substrate abstraction between the CK core and physical substrates.	§0.1
PhysicsThreatAlert	The CK system for detecting, classifying, routing, and responding to physics-level substrate threats.	§0.1
PostLattice_v2	Post-Quantum Lattice Substrate, Version 2. A formally defined CK substrate operating on classical hardware using lattice-based cryptographic primitives.	§5.2
PROPOSER	A CK runtime instance authorized to propose epoch state transitions in the consensus mechanism.	§8.2
Quorum	The minimum proportion of VALIDATOR votes required for a consensus proposal to be accepted.	§8.5
QIGEM_v1	Quantum-Integrated Generalized Execution Medium, Version 1. A formally defined CK	§5.1

Term	Definition	First Used In Section
	substrate targeting gate-model quantum processing units.	
Ring-LWE	Ring Learning With Errors. A lattice-based hardness assumption used in PostLattice_v2 cryptographic design.	§5.2.3
Rollback	The deterministic restoration of epoch state to a prior valid checkpoint, reversing all state changes since the checkpoint.	§6.6
SAI	Substrate Abstraction Interface. The formal interface all substrate adapters MUST implement to register with the MSL.	§4.3
Statutory Law	The second level of Design Law, strongly binding but subject to formal waiver process.	§10.2
Substrate	A physical or logical execution environment registered with the MSL via the SAI. In CK v1.7: QIGEM_v1, PostLattice_v2.	§2.2
SUBSTRATE_HALT	The highest PhysicsThreatAlert severity level. Requires immediate cessation of all substrate operations and mandatory human-in-the-loop escalation.	§7.5
T1 (Relaxation Time)	The characteristic time for a qubit to spontaneously decay from the excited state $\|1\rangle$ to the ground state $\|0\rangle$.	§5.1.3
T2 (Dephasing Time)	The characteristic time for a qubit's quantum phase to decohere due to environmental noise, limiting the coherence window for computation.	§5.1.3
VALIDATOR	A CK runtime instance authorized to vote on consensus proposals after independent verification.	§8.2

Appendix E — References

148. **[RFC2119]** Bradner, S. (1997). "Key words for use in RFCs to Indicate Requirement Levels." RFC 2119, BCP 14. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc2119

149. **[NISTSP800-208]** National Institute of Standards and Technology (2020). "Recommendation for Stateful Hash-Based Signature Schemes." NIST Special Publication 800-208. U.S. Department of Commerce.

150. **[FIPS203]** National Institute of Standards and Technology (2024). "Module-Lattice-Based Key-Encapsulation Mechanism Standard." FIPS 203. U.S. Department of Commerce.

151. **[FIPS204]** National Institute of Standards and Technology (2024). "Module-Lattice-Based Digital Signature Standard." FIPS 204. U.S. Department of Commerce.

152. **[FIPS205]** National Institute of Standards and Technology (2024). "Stateless Hash-Based Digital Signature Standard." FIPS 205. U.S. Department of Commerce.

153. **[IEEE7000]** IEEE Standards Association (2021). "IEEE Standard Model Process for Addressing Ethical Concerns during System Design." IEEE 7000-2021.

154. **[ISO25010]** International Organization for Standardization (2023). "Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model." ISO/IEC 25010:2023.

155. **[PROVDM]** Moreau, L., Missier, P., et al. (2013). "PROV-DM: The PROV Data Model." W3C Recommendation. World Wide Web Consortium.

156. **[LWE]** Regev, O. (2005). "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography." Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC 2005). pp. 84–93.

157. **[RINGLWE]** Lyubashevsky, V., Peikert, C., Regev, O. (2010). "On Ideal Lattices and Learning with Errors over Rings." Proceedings of EUROCRYPT 2010. Lecture Notes in Computer Science, Vol. 6110. Springer.

158. **[PAXOS]** Lamport, L. (1998). "The Part-Time Parliament." ACM Transactions on Computer Systems 16(2): 133–169.

159. **[PBFT]** Castro, M., Liskov, B. (1999). "Practical Byzantine Fault Tolerance." Proceedings of the 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI 1999). pp. 173–186.

160. **[QUANTUMERROR]** Gottesman, D. (2010). "An Introduction to Quantum Error Correction and Fault-Tolerant Quantum Computation." Proceedings of Symposia in Applied Mathematics, Vol. 68. American Mathematical Society.

161. **[RFC4122]** Leach, P., Mealling, M., Salz, R. (2005). "A Universally Unique IDentifier (UUID) URN Namespace." RFC 4122. Internet Engineering Task Force.

162. **[ISO8601]** International Organization for Standardization (2019). "Date and time — Representations for information interchange — Part 1: Basic rules." ISO 8601-1:2019.
